ROS 2 Labs (Windows)

Seventeen hands-on ROS 2 (Jazzy) projects. **Every lab is a complete, separate project** in its own folder under `labs\`: its own environment (`pixi.toml`), source code, scripts, automated test and manual (`README.md`). You can do any lab on its own, in any order; nothing is shared between them except the tools you install once.

You do not need administrator rights to run a lab. Each lab folder has a `README-SETUP.md` with its own setup steps, and a `README.md` with the lab itself. Three things do need an administrator, once per computer, and only for the labs that use them: the C++ compiler (*Visual Studio Build Tools*), Docker Desktop, and the NVIDIA driver. Each `README-SETUP.md` says if yours needs one.

1. Install pixi (once per computer, no administrator rights)

Open **PowerShell** and run, then close and reopen PowerShell:

```
powershell -ExecutionPolicy Bypass -c "irm -useb https://pixi.sh/install.ps1 | iex"
```

2. Get the labs (no Git needed)

Keep the folder outside OneDrive, for example in your user folder:

```
cd $HOME
Invoke-WebRequest https://github.com/ursikigai/ros-labs/archive/refs/heads/main.zip -OutFile ros-labs.zip
Expand-Archive ros-labs.zip -DestinationPath .
Rename-Item ros-labs-main ros-labs
cd ros-labs
```

(With Git installed you can use `git clone https://github.com/ursikigai/ros-labs.git ros-labs` instead.)

3. Run a lab

```
cd labs\Lab-02-robot-model
```

Open that folder's `README-SETUP.md` once, then its `README.md` (or its PDF in `pdf\`) and follow it. Every manual starts with the same two commands, `pixi install` (downloads the lab's environment, minutes the first time) and `pixi run verify`, and then continues with the lab's own steps. The last step of each manual is a one-command automated check (`pixi run test-...`).

The labs

Folder (under `labs\`)	Project	Also needs	Time
Course: build the robot step by step			
`Lab-01-turtle-catcher`	Turtle Catcher: nodes, topics, services, parameters, launch	C++ compiler	45 min
`Lab-02-robot-model`	DiffBot model: URDF, xacro, TF, RViz2		45 min

Folder (under `Labs\`)	Project	Also needs	Time
`Lab-03-simulation-odometry`	Simulator and wheel odometry		90 min
`Lab-04-lidar-avoidance`	LiDAR obstacle avoidance		60 min
`Lab-05-autonomous-navigation`	A* planner + pure pursuit + actions		90 min
`Lab-06-nav2`	The real Nav2 stack		45 min
Original public repositories, built and run unmodified (downloaded by the lab at a pinned commit)			
`Lab-U1-packt-ros2-from-scratch`	Packt "ROS 2 from Scratch", chapter 9 (Python + C++)	C++ compiler	90 min
`Lab-U2-turtlesim`	turtlesim built from source	C++ compiler	45 min
`Lab-U3-urdf-tutorial`	the eight `urdf_tutorial` robots		45 min
`Lab-U4-nav2-custom-planner`	Your own planner inside Nav2	C++ compiler	60 min
`Lab-U5-gazebo-slam-docker`	Gazebo, SLAM and Nav2 in Docker	Docker Desktop	30 min
NVIDIA Isaac Sim (physics simulator)			
`Lab-I1-isaac-drive`	Drive in Isaac Sim from ROS 2	NVIDIA RTX GPU	45 min
`Lab-I2-isaac-lidar`	LiDAR in Isaac Sim + obstacle avoider	NVIDIA RTX GPU	30 min
`Lab-I3-isaac-camera`	Camera: follow a red cube	NVIDIA RTX GPU	45 min
`Lab-I4-isaac-waypoints`	Waypoints	NVIDIA RTX GPU	30 min
`Lab-I5-isaac-line-follower`	Line follower	NVIDIA RTX GPU	30 min
`Lab-I6-isaac-ramp-imu`	Ramp and IMU	NVIDIA RTX GPU	30 min

"C++ compiler" = Visual Studio Build Tools with the *Desktop development with C++* workload (includes a Windows SDK). Labs without it are pure Python. The Isaac Sim labs need about 15 GB of disk for Isaac Sim, installed once into `C:\ix` and shared by all six.

Good to know

- **Where things go.** Each lab's ROS environment is kept in `C:\rlabs` and its build files in `C:\rb`, not inside the cloned folder (this avoids Windows' 260-character path limit and lets you move or zip a lab folder freely). pixi shares downloaded packages between labs, so the second lab is much quicker than the first.
- **One lab at a time.** Labs use the same ROS network; stop everything (Ctrl+C) before starting another lab. To clear stuck programs: `Get-Process | Where-Object { $_.Path -like "C:\rlabs\*" -or $_.Path -like "C:\ix\*" } | Stop-Process -Force`.

- **Red text in PowerShell** that starts with `Pixi task (...)` is pixi's status line, not an error.
- **Self-test.** Each lab can replay its own manual: `powershell -NoProfile -File tools\replay.ps1 README.md` (from inside the lab folder; it opens windows, so leave the mouse alone).
- **PDFs.** `pdf\` has every manual as a PDF and `ROS2-Labs-All.pdf` with all of them (rebuild with `python tools\make_pdfs.py` after `pip install markdown`).
- **What was tested, and what was not:** see TESTED.md.

Lab 1 - Turtle Catcher (nodes, topics, services, parameters, launch)

Aim. Build a game in which one node keeps dropping new turtles into the turtlesim window and another node drives `turtle1` to the nearest one and "catches" it. Start the nodes one at a time, inspect every ROS 2 communication type involved, catch turtles by hand through a service, then start everything with one launch file and change the game with a parameter.

Time. 45 min. **Needs.** Only this folder, plus a C++ compiler (Visual Studio Build Tools with the C++ workload and a Windows SDK) because the custom messages are compiled.

This folder is a complete project

Everything this lab needs is in this folder. Nothing from the other labs is required.

Requirements: Windows 10/11, PowerShell, pixi (no administrator rights needed; install steps are in README-SETUP.md), and Visual Studio Build Tools (C++ workload) and a Windows 10/11 SDK, for the compiled message package.

In this folder	
`ws\src\`	the ROS 2 packages; `ws\install\` appears after `pixi run build`
`scripts\`	helper scripts and the automated tests
`pixi.toml`	the environment and every `pixi run ...` command; `README.md` this manual; `tools\replay.ps1` re-runs the manual's commands as a self-test

Set it up once. `pixi install` puts ROS 2 Jazzy into `C:\rlabs` (a few GB; pixi's package cache makes the second lab on the same machine fast). Open PowerShell in this folder and run:

```
pixi install
pixi run verify
pixi run build
```

(`pixi run build` compiles this lab's packages into `ws\install`; run it again after every edit in `ws\src`.)

Run one lab at a time, and everything from this folder.

Theory

ROS 2 idea	Where it appears in this lab
Node - one program, one job	`turtlesim` (the window), `turtle_spawner`, `turtle_controller`
Topic - continuous one-way data	`/turtle1/pose` (where is it), `/turtle1/cmd_vel` (how to move), `/alive_turtles` (who is on screen)
Service - one request, one answer	`/spawn`, `/kill` (turtlesim) and our own `/catch_turtle`
Parameter - a setting read at start-up	`spawn_frequency`, `catch_closest_turtle_first`
Custom interface - your own message / service types	`Turtle.msg`, `TurtleArray.msg`, `CatchTurtle.srv` in `turtle_catcher_interfaces`

ROS 2 idea	Where it appears in this lab
Launch file - starts several nodes together	`turtle_catcher.launch.py`

```
turtlesim --/turtle1/pose--> turtle_controller --/turtle1/cmd_vel--> turtlesim
turtle_spawner --/spawn--> turtlesim           turtle_controller --/catch_turtle--> turtle_spawner
--/kill--> turtlesim
turtle_spawner --/alive_turtles--> turtle_controller
```

Control law (`logic.py`): with distance `d` and heading error `e` to the target, `v = min(2 d, 3)` and `w = 6 e`; if `|e| > 60 deg` the turtle first turns on the spot (`v = 0`). A turtle is caught when `d < 0.5`.

Procedure

1. Build (also needed after every edit in `ws\src`).

```
pixi run build
```

2. Look at the custom interface. A service has a request part and a reply part, separated by `---`.

```
pixi run ros interface show turtle_catcher_interfaces/srv/CatchTurtle
pixi run ros interface show turtle_catcher_interfaces/msg/TurtleArray
```

Part A - the nodes one at a time

3. T1 - the window. T2 - the spawner. A turtlesim window opens; from now on a new turtle appears every second and nobody removes it.

```
pixi run ros run turtlesim turtlesim_node
```

```
pixi run ros run turtle_catcher turtle_spawner
```

4. T3 - inspect.

```
pixi run ros node list --no-daemon --spin-time 5
pixi run ros topic list --no-daemon --spin-time 5
```

The spawner publishes the list of turtles on the screen (names `turtleN`; the highest N tells how many were spawned) and reads its settings from parameters:

```
pixi run ros topic echo /alive_turtles --no-daemon --once --spin-time 5
pixi run ros param get /turtle_spawner spawn_frequency --no-daemon --spin-time 5
```

5. T3 - call the service by hand. An unknown name must be refused (`success=False`). Then catch five turtles from the list yourself: each call that names a living turtle answers `success=True` and the turtle disappears from the window.

```
pixi run ros service call /catch_turtle turtle_catcher_interfaces/srv/CatchTurtle "{name: nope}"
$names = (pixi run ros topic echo /alive_turtles --no-daemon --once --spin-time 5 | Select-String
'name: (\w+)').Matches | ForEach-Object { $_.Groups[1].Value } | Select-Object -First 5
foreach ($n in $names) { "$n  " + (pixi run ros service call /catch_turtle
turtle_catcher_interfaces/srv/CatchTurtle "{name: $n}" | Select-String 'success=\w+').Matches.Value
}
```

Part B - the whole game from the launch file

6. Stop T1 and T2 (Ctrl+C), then T1 - start all three nodes (turtlesim, spawner, controller) with the settings from `config\turtle_catcher.yaml`. Now `turtle1` chases the nearest turtle and catches it by calling the same service you just used.

```
# Ctrl+C in T1 and T2
```

```
pixi run ros launch turtle_catcher turtle_catcher.launch.py
```

7. T3 - how many turtles are alive? This line waits 60 s, then prints how many turtles are alive and how many were spawned so far.

```
Start-Sleep 60
$t = pixi run ros topic echo /alive_turtles --no-daemon --once --spin-time 5 | Out-String; $m =
[regex]::Matches($t,'name: turtle(\d+)'); "alive: $($m.Count)   spawned: $(($m | ForEach-Object
{[int]$_.Groups[1].Value} | Measure-Object -Maximum).Maximum)"
```

8. Experiment: spawn faster. Stop T1 (Ctrl+C). Set `spawn_frequency` to 3.0 in the YAML file, rebuild (the installed copy is what runs), relaunch, wait 60 s, measure again with the line of step 7.

```
# Ctrl+C in T1
```

```
(Get-Content ws\src\turtle_catcher\config\turtle_catcher.yaml) -replace 'spawn_frequency:
1.0','spawn_frequency: 3.0' | Set-Content ws\src\turtle_catcher\config\turtle_catcher.yaml
pixi run build
```

```
pixi run ros launch turtle_catcher turtle_catcher.launch.py
```

```
Start-Sleep 60
$t = pixi run ros topic echo /alive_turtles --no-daemon --once --spin-time 5 | Out-String; $m =
[regex]::Matches($t,'name: turtle(\d+)'); "alive: $($m.Count)   spawned: $(($m | ForEach-Object
{[int]$_.Groups[1].Value} | Measure-Object -Maximum).Maximum)"
```

Put the file back when you are done (Ctrl+C in T1 first):

```
# Ctrl+C in T1
```

```
(Get-Content ws\src\turtle_catcher\config\turtle_catcher.yaml) -replace 'spawn_frequency: 3.0','spawn_frequency: 1.0' | Set-Content ws\src\turtle_catcher\config\turtle_catcher.yaml
pixi run build
```

Observations (a recorded run; yours differ slightly because spawn positions are random)

`spawn_frequency`	turtles alive 60 s after the launch	what it means
1.0 (default)	4-8 (recorded: 4 alive, 70 spawned)	the controller catches turtles as fast as they appear (about 1 per second)
3.0	about 50-60 (recorded: 61 alive, 212 spawned)	spawning outpaces catching, so the screen fills up

In Part A (no controller) every `catch_turtle` call with a listed name returned `success=True`; with `nope` it returned `success=False`.

Verification

```
pixi run test-class1
```

It starts the real launch file (turtlesim offscreen, no window) for 60 s and requires: the stack comes up, an unknown name is rejected, at least 30 turtles spawned, at least 5 caught, fewer than 15 alive, and `turtle1` travelled more than 10 units. Recorded: 6/6 checks (61 spawned, 54 caught, at most 9 alive, 134 units travelled).

Questions

1. Which topic would you echo to see where `turtle1` is? Which one to make it move? Who publishes each?
2. Why is `/catch_turtle` a service and `/alive_turtles` a topic?
3. In `logic.py`, why is the linear speed set to 0 when the heading error exceeds 60 degrees?
4. At `spawn_frequency: 3.0` the alive list grows. Which parameter or constant would you change so the controller keeps up?
5. *(Extension)* Add a `catch_count` topic that the controller publishes after every catch, and watch it with `ros topic echo`.

If it does not work

Symptom	Fix
`pixi run build` fails with `rc.exe` / `kernel32.lib` not found	install the Windows SDK (see "This folder is a complete project")
`package 'turtle_catcher' not found`	run `pixi run build`
`ros topic echo` says "does not appear to be published yet"	add `--spin-time 5`
No turtlesim window appears	wait 30 s on a cold start; `ros node list` should show `/turtlesim`
`success=False` for a name from the list in Part B	the controller caught it first - that is the game working

Lab 2 - Robot Model: URDF, xacro, TF and RViz

Aim. Describe a differential-drive robot (**DiffBot**) in a URDF file written with xacro, publish it with `robot_state_publisher`, see it in RViz2 and read its TF tree.

Time. 45 min. **Needs.** Only this folder (no compiler: the package is pure Python).

This folder is a complete project

Everything this lab needs is in this folder. Nothing from the other labs is required.

Requirements: Windows 10/11, PowerShell, pixi (no administrator rights needed; install steps are in README-SETUP.md), and nothing else.

In this folder	
`ws\src\`	the ROS 2 packages; `ws\install\` appears after `pixi run build`
`scripts\`	helper scripts and the automated tests
`pixi.toml`	the environment and every `pixi run ...` command; `README.md` this manual; `tools\replay.ps1` re-runs the manual's commands as a self-test

Set it up once. `pixi install` puts ROS 2 Jazzy into `C:\rlabs` (a few GB; pixi's package cache makes the second lab on the same machine fast). Open PowerShell in this folder and run:

```
pixi install
pixi run verify
pixi run build
```

(`pixi run build` compiles this lab's packages into `ws\install`; run it again after every edit in `ws\src`.)

Run one lab at a time, and everything from this folder.

Theory

- A **URDF** is an XML description: **links** (rigid bodies, with shape, colour, mass) joined by **joints** (parent -> child, with an origin offset and a type). Joint types used here: `fixed` (never moves) and `continuous` (rotates for ever - the wheels).
- **xacro** adds constants (`xacro:property`), arithmetic (`${wheel_separation / 2}`) and macros (`xacro:macro`) so the file has no repeated numbers. DiffBot defines `wheel_radius = 0.05 m`, `wheel_separation = 0.30 m` once; a robot simulator or odometry node written for this robot must use the same numbers (the unit test below checks the wheel position).
- **TF** is ROS's tree of coordinate frames. `robot_state_publisher` reads the URDF from the parameter `robot_description` and the wheel angles from `/joint_states`, and publishes one frame per link: fixed joints on `/tf_static` (once), moving joints on `/tf` (continuously).

```
base_footprint --fixed (z = +0.05, one wheel radius)--> base_link --continuous--> left_wheel_link
(y = +0.15)
                                                                  --continuous--> right_wheel_link
(y = -0.15)
                                                                  --fixed-------> caster_link,
lidar_link (x = +0.08, z = +0.07)
```

Procedure

1. Expand and check the model. xacro turns the macro file into plain URDF; `check_urdf` parses it and prints the link tree.

```
pixi run build-core
pixi run xacro ws/src/diffbot_description/urdf/diffbot.urdf.xacro -o diffbot.urdf
pixi run check_urdf diffbot.urdf
Remove-Item diffbot.urdf
```

2. T1 - show the robot. RViz2 and a *Joint State Publisher* window with two sliders open. You should see a blue box with two black wheels, a grey caster and a red lidar disc. Drag the `left_wheel_joint` slider: the left wheel spins.

```
pixi run demo-model
```

3. T2 - what is being published.

```
pixi run ros node list --no-daemon --spin-time 5
pixi run ros topic list --no-daemon --spin-time 5
pixi run ros topic echo /joint_states --no-daemon --once --spin-time 5
```

4. T2 - read a transform. `tf2_echo <parent> <child>` prints where the child frame is relative to the parent. Stop it with Ctrl+C. First a fixed joint, then a wheel joint (drag the slider while it runs: the rotation changes, the translation does not).

```
pixi run ros run tf2_ros tf2_echo base_footprint lidar_link
```

```
pixi run ros run tf2_ros tf2_echo base_link left_wheel_link
```

The lidar is at z = 0.12 m = 0.05 (wheel radius) + 0.05 (half the body height) + 0.02. The left wheel is 0.15 m to the left (= separation / 2).

5. T2 - draw the whole tree. Writes `frames_<time>.pdf` into the current folder (open it; delete it afterwards).

```
pixi run ros run tf2_tools view_frames
Remove-Item frames_*
```

6. Break it on purpose. Widen the robot: change `wheel_separation` from 0.30 to 0.40 in the xacro. The unit test that pins the wheel positions (+-0.15 m) notices:

```
(Get-Content ws\src\diffbot_description\urdf\diffbot.urdf.xacro) -replace 'name="wheel_separation"
value="0.30"','name="wheel_separation" value="0.40"' | Set-Content
ws\src\diffbot_description\urdf\diffbot.urdf.xacro
pixi run test-unit
```

Undo it:

```
(Get-Content ws\src\diffbot_description\urdf\diffbot.urdf.xacro) -replace 'name="wheel_separation"
value="0.40"','name="wheel_separation" value="0.30"' | Set-Content
ws\src\diffbot_description\urdf\diffbot.urdf.xacro
pixi run test-unit
```

Observations (recorded)

Quantity	Value
Links / joints	6 links (`base_footprint` , `base_link` , 2 wheels, caster, lidar) and 5 joints
`base_footprint` -> `lidar_link`	(0.080, 0.000, 0.120)
`base_link` -> `left_wheel_link`	(0.000, 0.150, 0.000)
`/joint_states` names	`left_wheel_joint` , `right_wheel_joint` (only the *continuous* joints need a value)

Verification

```
pixi run test-class2
```

Checks: `/robot_description` is published and names `diffbot` ; the fixed TF `base_footprint -> lidar_link` resolves; after joint states arrive, `base_link -> left_wheel_link` has y = 0.15 m; RViz2 stays alive for 25 s. (The RViz2 part runs without a screen, so it proves only that the launch does not crash. The picture in step 2 is the real check.)

Questions

1. What would change in the TF tree if `lidar_joint` were `continuous` instead of `fixed` ? Which topic would you then have to feed?
2. Why is `base_footprint` the root and `base_link` raised by one wheel radius?
3. Add a camera link 0.12 m in front of the lidar. Which unit test (if any) fails, and why or why not? (Read `ws\src\diffbot_description\test\test_urdf.py` .)
4. Which value in the xacro would you change to make the robot wider, and what else in the course must then change?

If it does not work

Symptom	Fix
RViz shows nothing / grey	set *Fixed Frame* to `base_footprint` ; wait for the model to load
`error creating render window`	you are in offscreen mode (`QT_QPA_PLATFORM`); open a normal PowerShell

Symptom	Fix
`tf2_echo` waits for ever	`demo-model` is not running

Lab 3 - Simulation and Odometry

Aim. Drive DiffBot in a small simulator, write the **wheel-odometry** node yourself, and compare your estimate with the simulator's ground truth. Then make the wheels imperfect and watch odometry drift.

Time. 90 min. **Needs.** Only this folder (no compiler).

This folder is a complete project

Everything this lab needs is in this folder. Nothing from the other labs is required.

Requirements: Windows 10/11, PowerShell, pixi (no administrator rights needed; install steps are in README-SETUP.md), and nothing else.

In this folder	
`ws\src\`	the ROS 2 packages; `ws\install\` appears after `pixi run build`
`scripts\`	helper scripts and the automated tests
`code\`	reference solutions / helper programs for this lab
`pixi.toml`	the environment and every `pixi run ...` command; `README.md` this manual; `tools\replay.ps1` re-runs the manual's commands as a self-test

Set it up once. `pixi install` puts ROS 2 Jazzy into `C:\rlabs` (a few GB; pixi's package cache makes the second lab on the same machine fast). Open PowerShell in this folder and run:

```
pixi install
pixi run verify
pixi run build
```

(`pixi run build` compiles this lab's packages into `ws\install`; run it again after every edit in `ws\src`.)

Run one lab at a time, and everything from this folder.

Theory

`diffbot_sim` is a kinematic simulator (no inertia, no friction) in a 10 m x 8 m room. It reads `/cmd_vel`, moves the robot, and publishes what a real robot would: `/joint_states` (wheel angles), `/scan` (360-ray lidar), `/map` and, for checking only, `/ground_truth`. It stops the robot 0.5 s after the last command (watchdog), so a command must be repeated.

With wheel radius `r = 0.05 m` and wheel separation `L = 0.30 m`:

Direction	Formula
command -> wheel speeds	`w_left = (v - w L/2) / r`, `w_right = (v + w L/2) / r`
wheel angle change -> distance	`d_left = r * delta_phi_left`, `d_right = r * delta_phi_right`
distance -> robot motion	`ds = (d_left + d_right) / 2`, `dtheta = (d_right - d_left) / L`
position update (midpoint heading)	`x += ds cos(theta + dtheta/2)`, `y += ds sin(theta + dtheta/2)`, `theta += dtheta`

Frames: `map --(static, = start pose)--> odom --(odometry node)--> base_footprint -> base_link -> wheels, lidar_link`. `/odom` starts at (0, 0) in frame `odom`; `/ground_truth` is in frame `map`, where the robot starts at **(1, 1)**. So *truth = odom + (1, 1)* when odometry is perfect. Heading from a quaternion (z, w): `yaw = 2 * atan2(z, w)`.

Procedure

1. T1 - start the simulator (RViz2 opens: robot, laser points, room walls).

```
pixi run demo-sim
```

2. T2 - what the simulator publishes.

```
pixi run ros node list --no-daemon --spin-time 5
pixi run ros topic list --no-daemon --spin-time 5
```

3. T3 - your own odometry (the exercise). Open `ws\src\diffbot_odometry\exercises\odometry_template.py` and complete the four `TODO`s using the formulas above (the solution is at the end of this page). Run it; it publishes `/my_odom`. With the TODOs still empty it runs without errors and publishes nothing. Here the finished reference file is used:

```
pixi run python code\odometry_solution.py
```

4. T2 - drive. The three commands are: forward 0.8 m, turn left about 1.5 rad, forward 0.8 m (arguments: linear m/s, angular rad/s, seconds).

```
pixi run drive 0.2 0.0 4
pixi run drive 0.0 0.5 3
pixi run drive 0.2 0.0 4
```

5. T2 - compare the three poses. Compute `yaw = 2 atan2(z, w)` from each orientation.

```
pixi run ros topic echo /odom --no-daemon --once --spin-time 4 --field pose.pose
pixi run ros topic echo /my_odom --no-daemon --once --spin-time 4 --field pose.pose
pixi run ros topic echo /ground_truth --no-daemon --once --spin-time 4 --field pose.pose
```

Expected: `/odom` and `/my_odom` identical; `/ground_truth` = the same plus (1, 1).

6. Break the odometry on purpose. Stop T1 and T3 with Ctrl+C.

```
# Ctrl+C in T1 and T3
```

Restart T1 with a wheel that is really 3 % off. The robot now moves with the imperfect wheel, but the encoders (`/joint_states`) still report the ideal rotation - exactly like a real robot. Repeat steps 4 and 5 (without T3).

```
pixi run ros launch diffbot_bringup sim.launch.py rviz:=true wheel_bias:=0.03
```

```
pixi run drive 0.2 0.0 4
pixi run drive 0.0 0.5 3
pixi run drive 0.2 0.0 4
```

```
pixi run ros topic echo /odom --no-daemon --once --spin-time 4 --field pose.pose
pixi run ros topic echo /ground_truth --no-daemon --once --spin-time 4 --field pose.pose
```

Error = distance between `/odom` (x, y) and `/ground_truth` minus (1, 1), and the difference of yaw. Try `wheel_noise:=0.05` (random slip) instead of `wheel_bias:=0.03` too.

Observations (recorded runs of the same three drives; yours differ by a few mm)

Setting	odometry says (x, y, yaw)	truth - (1, 1)	error
perfect wheels	0.844, 0.799, 1.516	0.844, 0.799, 1.516	0.000 m, 0.000 rad
`wheel_noise:=0.05` (random slip)	0.844, 0.799, 1.515	0.857, 0.788, 1.483	0.017 m, 0.032 rad
`wheel_bias:=0.03` (right wheel 3 % off)	0.837, 0.777, 1.523	0.739, 0.821, 1.704	0.107 m, 0.181 rad

Random errors partly cancel; a constant error does not. That is why real robots fuse odometry with other sensors.

Verification

```
pixi run test-class3
```

15 checks: `/joint_states`, `/scan` (360 rays), `/odom`, `/ground_truth` publish; the TF chain `map -> odom -> base_footprint -> base_link -> lidar_link` resolves; after 0.8 m forward and a 1.5 rad turn the odometry is within 3 cm and 0.03 rad of the truth; the robot stops by itself when commands stop.

Solution to the exercise (read after you tried)

```
d_left  = (left  - self.prev_left)  * WHEEL_RADIUS       # TODO 1
d_right = (right - self.prev_right) * WHEEL_RADIUS
ds      = (d_left + d_right) / 2.0                        # TODO 2
dtheta  = (d_right - d_left) / WHEEL_SEPARATION
self.x += ds * math.cos(self.theta + dtheta / 2.0)        # TODO 3
self.y += ds * math.sin(self.theta + dtheta / 2.0)
self.theta += dtheta
# TODO 4: Odometry(); header.frame_id "odom"; child_frame_id "base_footprint"; position x, y;
#         orientation z = sin(theta/2), w = cos(theta/2); self.pub.publish(odom)
```

The complete tested file is `code\odometry_solution.py`.

Questions

1. Why does the simulator need the 0.5 s watchdog and why does `drive` publish at 20 Hz?
2. In the table, which error is larger for the same 3 % imperfection: position or heading? Why does a heading error make later position errors worse?

3. Replace the midpoint heading by plain Euler (`x += ds*cos(theta)`). How much worse is the result after driving a full circle?
4. Why is `map -> odom` a *static* transform here, and what would normally provide it on a real robot?

If it does not work

Symptom	Fix
`drive` prints "nothing is listening on /cmd_vel"	T1 is not running yet (wait 20-30 s)
Robot moves less than expected	the drive started before ROS discovery finished; just repeat
`/my_odom` stays empty	TODOs not complete, or T1 not running; `ros topic list` should show `/joint_states`

Lab 4 - LiDAR and Reactive Obstacle Avoidance

Aim. Make DiffBot wander the room without touching a wall, using only its 2D LiDAR and a small rule-based controller (no map, no goal).

Time. 60 min. **Needs.** Only this folder (no compiler).

This folder is a complete project

Everything this lab needs is in this folder. Nothing from the other labs is required.

Requirements: Windows 10/11, PowerShell, pixi (no administrator rights needed; install steps are in README-SETUP.md), and nothing else.

In this folder	
`ws\src\`	the ROS 2 packages; `ws\install\` appears after `pixi run build`
`scripts\`	helper scripts and the automated tests
`pixi.toml`	the environment and every `pixi run ...` command; `README.md` this manual; `tools\replay.ps1` re-runs the manual's commands as a self-test

Set it up once. `pixi install` puts ROS 2 Jazzy into `C:\rlabs` (a few GB; pixi's package cache makes the second lab on the same machine fast). Open PowerShell in this folder and run:

```
pixi install
pixi run verify
pixi run build
```

(`pixi run build` compiles this lab's packages into `ws\install`; run it again after every edit in `ws\src`.)

Run one lab at a time, and everything from this folder.

Theory

A `sensor_msgs/LaserScan` is a list of distances, one per ray, starting at `angle_min` and stepping by `angle_increment`. In the simulator: 360 rays, 1 degree apart, from -180 to +179 degrees (0 degrees = straight ahead), range 0.12-6.0 m; a ray that hits nothing is `inf`. The scan is in frame `lidar_link`, 8 cm ahead of the robot centre.

The controller (`diffbot_navigation/reactive.py`, pure Python) splits the scan into sectors - **front** (+-30 deg, minimum range) and **left** / **right** (30-90 deg, mean range) - and applies three rules:

Situation	Action
front < 0.45 m (`STOP_DIST`)	stop, turn on the spot towards the side with more room, and **remember the direction**
0.45 m < front < 0.90 m (`SLOW_DIST`)	slow down; keep turning if already turning, otherwise nudge away from the nearer side
front > 0.90 m	cruise at 0.25 m/s

Remembering the turn direction (`turn_dir`) stops the robot dithering left/right when both sides look equally open. `inf` / `nan` ranges are first replaced by 10 m, otherwise a single `inf` would make a mean infinite.

Procedure

1. T1 - start the robot with its avoider (RViz2 shows the room, the robot and the red laser points; the robot starts moving by itself).

```
pixi run demo-avoid
```

2. T2 - look at the data and the decision.

```
pixi run ros node list --no-daemon --spin-time 5
pixi run ros topic list --no-daemon --spin-time 5
pixi run ros topic echo /scan --no-daemon --once --spin-time 4 --truncate-length 6
pixi run ros topic echo /cmd_vel --no-daemon --once --spin-time 4
```

3. T2 - how often does the lidar publish? (stops itself after 15 s)

```
pixi run ros topic hz /scan --spin-time 5
```

4. T2 - is it exploring? Print the robot's true position five times, 6 s apart; the numbers should keep changing, within x 0-10 and y 0-8.

```
1..5 | ForEach-Object { pixi run ros topic echo /ground_truth --no-daemon --once --spin-time 4 --
field pose.pose.position; Start-Sleep 6 }
```

5. T2 - collisions. `/collision_count` publishes only when the robot touches something. 20 s of silence = no collision.

```
pixi run ros topic echo /collision_count --no-daemon --spin-time 5
```

6. Experiment: make the robot braver. In `ws\src\diffbot_navigation\diffbot_navigation\reactive.py` the robot stops turning on the spot 0.45 m in front of a wall. The robot's radius is 0.17 m and the lidar sits 0.08 m ahead of the centre. Reduce `STOP_DIST` to 0.25 and run the automated test (stop T1 with Ctrl+C first). Predict first: will it collide?

```
# Ctrl+C in T1
```

```
(Get-Content ws\src\diffbot_navigation\diffbot_navigation\reactive.py) -replace 'STOP_DIST =
0.45','STOP_DIST = 0.25' | Set-Content ws\src\diffbot_navigation\diffbot_navigation\reactive.py
pixi run build-core
pixi run test-class4
```

Record the result (`PASS` / `FAIL` lines) in the table below, then try 0.15 and 0.08 the same way. Restore the value when you are done:

```
(Get-Content ws\src\diffbot_navigation\diffbot_navigation\reactive.py) -replace 'STOP_DIST = 0.25','STOP_DIST = 0.45' | Set-Content ws\src\diffbot_navigation\diffbot_navigation\reactive.py
pixi run build-core
```

Observations (recorded)

Quantity	Value
`/scan`	360 rays, `angle_increment` 0.01745 rad, `range_max` 6.0, 10.0 Hz
Typical `/cmd_vel` in open space	`linear.x = 0.25`, `angular.z = 0`
60 s of wandering with `STOP_DIST = 0.45`	13.8 m driven, 14 different 1 m x 1 m cells visited, 0 collisions
`STOP_DIST = 0.25`	14.1 m driven, 15 cells, **0 collisions** - passes

Verification

```
pixi run test-class4
```

The test runs the avoider for 60 s and requires: more than 8 m driven, at least 4 different cells visited, and no collision. The controller's rules are also unit-tested: read `ws\src\diffbot_navigation\test\test_navigation_logic.py` and run `pixi run test-unit`.

Questions

1. Predict `compute_command` for: (a) open space, (b) a wall dead ahead at 0.3 m with more room on the left, (c) all ranges `inf`. Check your answers in the unit tests.
2. Why must `inf` be replaced before the mean is computed?
3. Why does the controller keep `turn_dir` instead of recomputing left/right every time?
4. The robot visits only 14 of 80 cells in a minute. How would you make it explore more systematically?
5. The test still passes with `STOP_DIST = 0.08`, although the robot radius is 0.17 m. Which of the three rules is keeping the robot away from the walls then? What would you change (and which constant) to make the test fail? `r`n6. *(Extension)* Write a right-hand wall follower that keeps 0.4 m from the wall; add a bumper recovery that reverses for 1 s when `/collision_count` increases.

If it does not work

Symptom	Fix
Robot does not move	the avoider needs the simulator's `/scan`; `ros node list` must show `/diffbot_sim` and `/obstacle_avoider`
A Python edit has no effect	run `pixi run build-core`
`topic hz` prints nothing	add `--spin-time 5` and wait 10 s

Lab 5 - Autonomous Navigation (A* planner + pure pursuit)

Aim. Give DiffBot a goal anywhere on the map. It plans a collision-free path around the walls and drives there on its own. You click a goal in RViz2 or send it from the command line.

Time. 90 min. **Needs.** Only this folder (no compiler).

This folder is a complete project

Everything this lab needs is in this folder. Nothing from the other labs is required.

Requirements: Windows 10/11, PowerShell, pixi (no administrator rights needed; install steps are in README-SETUP.md), and nothing else.

In this folder	
`ws\src\`	the ROS 2 packages; `ws\install\` appears after `pixi run build`
`scripts\`	helper scripts and the automated tests
`pixi.toml`	the environment and every `pixi run ...` command; `README.md` this manual; `tools\replay.ps1` re-runs the manual's commands as a self-test

Set it up once. `pixi install` puts ROS 2 Jazzy into `C:\rlabs` (a few GB; pixi's package cache makes the second lab on the same machine fast). Open PowerShell in this folder and run:

```
pixi install
pixi run verify
pixi run build
```

(`pixi run build` compiles this lab's packages into `ws\install`; run it again after every edit in `ws\src`.)

Run one lab at a time, and everything from this folder.

Theory

Navigation has three parts:

Part	Node	What it does
Global planner - *where* to go	`planner_server`	inflates the walls by the robot radius + margin (0.30 m), finds the shortest path with **A*** (8-connected, no corner cutting), shortens it ("string pulling"), resamples it every 0.1 m, publishes `/plan`
Controller - *how* to follow it	`pure_pursuit` (inside `navigator`)	steers towards the point 0.35 m ahead on the path; slows near the goal; turns on the spot when facing away
Executive - supervise	`navigator`	serves the **action** `navigate_to_pose`, asks the planner for a path (service `plan_path`), reads the robot pose from TF (`map -> base_footprint`), sends `/cmd_vel`, reports progress, aborts when stuck

An **action** is a service with feedback and cancel: the client sends a goal, gets `distance_remaining` while it runs and a result at the end. Result codes of this navigator: 0 success, 1 no plan, 2 no pose (TF), 3 blocked / stuck, 4 timeout.

Map (frame `map`, metres): room 10 x 8; wall 1 at x = 3.0-3.2 (y 0-5); wall 2 at x = 6.0-6.2 (y 3-8); a box at (4.4-5.2, 5.5-6.3); robot starts at (1, 1). Going from (1, 1) to (8.5, 6.5) therefore needs an "S": up around wall 1, down and around wall 2.

Procedure

1. T1 - start the whole stack (simulator, planner, navigator, RViz2). Cold start takes 30-40 s on Windows.

```
pixi run demo-nav
```

2. T2 - look at what is running. The navigator and planner are nodes; `plan_path` is the service between them; `interface show` shows the goal / result / feedback fields of the action; the planner's inflation is a parameter.

```
pixi run ros node list --no-daemon --spin-time 5
pixi run ros service list --spin-time 5
pixi run ros interface show nav2_msgs/action/NavigateToPose
pixi run ros param get /planner_server inflation_radius --no-daemon --spin-time 5
```

3. In RViz2 - click a goal. Choose the **Nav2 Goal** tool in the top toolbar and click a free spot, for example near the top-right of the room. The green line is the planned path; the robot follows it. (A click inside a wall is rejected.)

```
# a human clicks in the RViz2 window
```

4. T2 - the same from the command line. `goto` sends the action goal and prints progress every 2 s; the exit code is 0 for "reached". It takes about a minute.

```
pixi run goto 8.5 6.5
```

5. T2 - a goal inside a wall. (3.1, 2.0) is inside wall 1. The planner finds no path and the navigator aborts with code 1.

```
pixi run goto 3.1 2.0
```

6. T2 - check that nothing was hit. `/collision_count` publishes only on a collision, so 15 s of silence is the pass condition.

```
pixi run ros topic echo /collision_count --no-daemon --spin-time 5
```

7. Experiment: shrink the safety margin. The robot radius is 0.17 m. With only 0.22 m inflation there is just 5 cm of slack for tracking error. Replace the planner node by one with `inflation_radius:=0.22` and send the same far goal. Stop T1 first, then:

```
# Ctrl+C in T1
```

```
pixi run demo-nav
```

Close the planner of that launch and start your own (T3):

```
Get-Process planner_server | Stop-Process -Force
pixi run ros run diffbot_navigation planner_server --ros-args -p inflation_radius:=0.22
```

```
pixi run goto 8.5 6.5
```

```
pixi run ros topic echo /collision_count --no-daemon --spin-time 5
```

Record in the table whether it reached the goal and whether `/collision_count` published anything. (Stop all terminals afterwards.)

```
# Ctrl+C in T1 and T3
```

Observations (recorded)

Quantity	Value
`goto 8.5 6.5`	REACHED, last feedback 0.15-0.3 m from the goal, about 59 s, path about 13 m (straight line 9.3 m)
`goto 3.1 2.0` (inside wall)	`FAILED: no plan - planner returned no path`, exit code 1
`/collision_count`	no message (0 collisions)
`inflation_radius` 0.30 (default)	`Double value is: 0.3`
`inflation_radius:=0.22`	planner log: `inflated by 5 cells` (instead of 6); the goal was still **reached** (last feedback 0.25 m), no collision message. The margin is smaller, not yet broken

Verification

```
pixi run test-class5
```

9 checks: the action server comes up; a goal at (8.5, 6.5) succeeds; the robot ends within 0.25 m of it; `/plan` was published; the robot detours (travelled more than 1.15 x the straight line); no collision; a goal in a wall is aborted with code 1. Recorded: reached 0.14 m from the goal, 13.2 m driven, 0 collisions.

Questions

1. Why is the planner given a larger "robot" than the real one (0.30 m instead of 0.17 m)? Which error does the extra margin pay for?
2. Why does the navigator need TF (`map -> base_footprint`), and which two earlier labs produce that transform?
3. The planner and the navigator talk through a *service*, the client and the navigator through an *action*. Why is each the right choice?
4. Write your own `plan_path` service node (for example Dijkstra, or a straight line). The navigator needs no change - what property of ROS 2 makes that possible?

5. *(Extension)* Make the navigator honour the goal's orientation; re-plan every 2 s so the robot can cope with a new obstacle.

If it does not work

Symptom	Fix
`goto` waits at "waiting for the navigator"	T1 is not running or still starting (wait 40 s)
`goto` says "goal rejected"	the navigator is not active yet; wait and retry
RViz2 map is blank	set *Fixed Frame* to `map`; wait for the map
Robot does not move	`pixi run ros node list --no-daemon --spin-time 5` must show `/diffbot_sim`, `/navigator`, `/planner_server`

Lab 6 - The Real Nav2 Stack on the Same Robot

Aim. Drive DiffBot around the walls with **Nav2**, the standard ROS 2 navigation framework, using a simulator, a robot and a `goto` tool that are all in this folder. Compare it with a hand-written planner + navigator (the separate lab *Autonomous Navigation*).

Time. 45 min. **Needs.** Only this folder (no compiler).

This folder is a complete project

Everything this lab needs is in this folder. Nothing from the other labs is required.

Requirements: Windows 10/11, PowerShell, pixi (no administrator rights needed; install steps are in README-SETUP.md), and nothing else.

In this folder	
`ws\src\`	the ROS 2 packages; `ws\install\` appears after `pixi run build`
`scripts\`	helper scripts and the automated tests
`pixi.toml`	the environment and every `pixi run ...` command; `README.md` this manual; `tools\replay.ps1` re-runs the manual's commands as a self-test

Set it up once. `pixi install` puts ROS 2 Jazzy into `C:\rlabs` (a few GB; pixi's package cache makes the second lab on the same machine fast). Open PowerShell in this folder and run:

```
pixi install
pixi run verify
pixi run build
```

(`pixi run build` compiles this lab's packages into `ws\install`; run it again after every edit in `ws\src`.)

Run one lab at a time, and everything from this folder.

Theory

Nav2 splits navigation into servers that are started and supervised by a **lifecycle manager** (each server moves through *unconfigured* -> *inactive* -> *active*). Nav2 offers the standard **action** `navigate_to_pose` (a hand-written navigator, as in the separate lab *Autonomous Navigation*, can offer the same one), so any client works with either.

A hand-written navigator (lab ***Autonomous Navigation***)	Nav2 (here, `nav2_diffbot.yaml`)
`planner_server` + A* on an inflated grid	`planner_server` with the **NavFn** plugin on a **global costmap** (static map + inflation layer)
pure pursuit inside `navigator`	`controller_server` with **Regulated Pure Pursuit** on a **local costmap** (lidar obstacles)
"blocked / stuck" abort	`behavior_server` (spin, back up, wait) and `bt_navigator` (a *behaviour tree* that decides what to try)
a few hundred lines of Python	C++ servers configured by one YAML file

The course launch file `nav2.launch.py` starts only the four servers a goal needs, because Nav2's stock `navigation_launch.py` stalled on this Windows setup.

Procedure

1. T1 - start the simulator and Nav2. Wait for the line `Managed nodes are active` (10-90 s; a goal sent earlier is rejected).

```
pixi run ros launch diffbot_bringup nav2.launch.py rviz:=true
```

2. T2 - the Nav2 nodes.

```
pixi run ros node list --no-daemon --spin-time 5
```

3. T2 - send the goal across the room. `goto` sends the `navigate_to_pose` action goal. Nav2 plans with NavFn and follows with Regulated Pure Pursuit.

```
pixi run goto 8.5 6.5
```

4. T2 - a goal inside a wall. A hand-written navigator may reject this. Nav2's planner accepts a goal inside an obstacle if there is a free cell nearby and drives to it.

```
pixi run goto 3.1 2.0
```

Compare the final position printed by `goto` with (3.1, 2.0). Stop T1 with Ctrl+C.

```
# Ctrl+C in T1
```

Observations (recorded)

	hand-written A* + pure pursuit (lab *Autonomous Navigation*, for reference)	**Nav2 (NavFn + Regulated Pure Pursuit), measured here**
Time to start	30-40 s	10-90 s until `Managed nodes are active`
`goto 8.5 6.5`	reached, about 13.2 m driven	reached, about 14.0 m driven, 0 collisions
`goto 3.1 2.0` (inside wall)	`FAILED: no plan`	reached: robot stopped near (3.5, 2.1), the nearest free point

Verification

```
pixi run test-nav2
```

8 checks: all Nav2 lifecycle nodes become active; `navigate_to_pose` is available; the goal is accepted and succeeds; the robot ends within 0.35 m; no collision; a plan was published on `/plan`; the robot detoured around both walls. Recorded: 14.0 m driven for a 9.3 m straight line, 0 collisions, plan published 57 times.

Questions

1. A simple navigator has a global planner, a path follower and an executive. Which Nav2 server corresponds to each?
2. Why does Nav2 need *two* costmaps (global and local)? What does each one see?
3. Why can the same `goto` script drive both stacks?
4. Open `ws\src\diffbot_bringup\config\nav2_diffbot.yaml`. Find the planner plugin, the controller plugin and `robot_radius`. What would you change to use a different planner?

If it does not work

Symptom	Fix
`goto` prints "goal rejected"	Nav2 is not active yet - wait for `Managed nodes are active` in T1
Nothing happens for minutes after the `route_server` line	you started Nav2's stock `navigation_launch.py`; use `nav2.launch.py`
`ros lifecycle get` says "Node not found"	that command has no discovery wait; read the T1 log instead

Lab I1 - Drive DiffBot in NVIDIA Isaac Sim from ROS 2

Aim. Load the course robot into **Isaac Sim**, a physics (PhysX) and ray-tracing simulator, and drive it with a standard `/cmd_vel` command. Read `/odom`, `/tf` and `/joint_states` and compare the result with a simple kinematic simulator.

Time. 45 min (+ 10-minute download once, and 3 minutes for the very first start). **Needs.** Only this folder, an **NVIDIA RTX GPU** (tested: RTX 5050 Laptop, 8 GB, driver 592.82) and about 15 GB of disk for Isaac Sim, which `pixi run isaac-install` downloads once for all Isaac labs on the machine. By running the scripts you accept NVIDIA's Isaac Sim licence.

This folder is a complete project

Everything this lab needs is in this folder. Nothing from the other labs is required.

Requirements: Windows 10/11, PowerShell, pixi (no administrator rights needed; install steps are in README-SETUP.md), and an NVIDIA RTX GPU with a current driver; about 15 GB disk for Isaac Sim (shared by all Isaac labs, installed once into `C:\ix`); internet for the first install.

In this folder	
`isaac\`	the Isaac Sim scene scripts and ROS 2 nodes
`scripts\`	helper scripts and the automated tests
`code\`	reference solutions / helper programs for this lab
`pixi.toml`	the environment and every `pixi run ...` command; `README.md` this manual; `tools\replay.ps1` re-runs the manual's commands as a self-test

Set it up once. `pixi install` puts ROS 2 Jazzy into `C:\rlabs` (a few GB; pixi's package cache makes the second lab on the same machine fast). Open PowerShell in this folder and run:

```
pixi install
pixi run verify
```

Run one lab at a time, and everything from this folder.

Theory

Isaac Sim needs Python 3.11, ROS 2 Jazzy on Windows needs 3.12, so they live in two environments and talk over DDS like two computers - both use ROS domain **78**. Isaac Sim's own ROS 2 bridge publishes and subscribes on its side.

```
Isaac Sim (isaac\, Python 3.11, installed in C:\ix)                ROS 2 (course environment,
Python 3.12)
 USD scene + PhysX physics + OmniGraph  <----- DDS, domain 78 ----->  pixi run isaac-ros ...,
isaac-drive ...
 /cmd_vel -> wheel joints ;  /odom /tf /joint_states /clock ->
```

- The robot is a **xacro** differential-drive model (expanded to `isaac\assets\diffbot.urdf`). Physics needed three additions that RViz does not: a 2 cm gap under the body so the wheels carry the weight, explicit link masses, and a centre of mass 3 cm behind the axle so the robot rests on its caster.
- An **OmniGraph** (Isaac's visual dataflow graph, built here from Python) is the whole driver: `/cmd_vel` -> `DifferentialController` (the standard differential-drive maths: `w_left = (v - w L/2)/r`, `w_right = (v + w L/2)/r`, r = 0.05, L = 0.30) -> wheel joints. The difference to Lab 3: here the **physics engine** moves the robot, so wheels can slip.
- Isaac Sim runs on **simulated time** (`/clock`), and not at wall-clock speed. The course helper `pixi run isaac-drive` counts *wall-clock* seconds, so the distance it drives depends on how fast your simulation runs (in window mode here: 1.32 m instead of 0.8 m). `code\drive_sim.py` counts `/clock` instead, so "0.2 m/s for 4 s" means 4 *simulated* seconds. Always pace experiments by `/clock`.

Procedure

1. Install Isaac Sim (once per machine, shared by all Isaac labs; about 10 GB into `C:\ix`, a short path because Windows' 260-character limit breaks Isaac's DLLs inside long folder names).

```
pixi run isaac-install
```

2. T1 - start the simulation with a window (first start about 3 min while shaders compile, later 15-60 s). Omit `--gui` to run without a window.

```
pixi run isaac-p1 --gui
```

3. T2 - what the simulator publishes. (`isaac-ros` is `ros2` on domain 78.)

```
pixi run isaac-ros topic list --no-daemon --spin-time 5
```

4. T2 - drive. Forward 0.2 m/s for 4 simulated seconds, then a left turn at 0.5 rad/s for 3 simulated seconds. Read the pose before and after each move.

```
pixi run isaac-ros topic echo /odom --no-daemon --once --spin-time 5 --field pose.pose
pixi run python code\drive_sim.py 0.2 0.0 4
pixi run isaac-ros topic echo /odom --no-daemon --once --spin-time 5 --field pose.pose
pixi run python code\drive_sim.py 0.0 0.5 3
pixi run isaac-ros topic echo /odom --no-daemon --once --spin-time 5 --field pose.pose
```

Compute the forward distance from the first two outputs and the heading change `2 * atan2(z, w)` from the last two. Expected: about 0.8 m (ideal: 0.2 x 4) and about 1.4-1.5 rad (ideal: 0.5 x 3 = 1.5). Physics makes it differ a little from the ideal (wheels need time to speed up and may slip); the automated test accepts 0.6-1.0 m and 1.1-1.9 rad.

For comparison, the wall-clock helper (`pixi run isaac-drive 0.2 0.0 4`) in window mode drove the robot 1.32 m and `isaac-drive 0.0 0.5 3` turned it 2.2 rad: the simulation ran about 1.5 times faster than real time.

5. T2 - the joints and the transform tree. The wheel angles and the frame `odom -> base_footprint` come from the simulator.

```
pixi run isaac-ros topic echo /joint_states --no-daemon --once --spin-time 5 --field name
pixi run isaac-ros topic echo /tf --no-daemon --once --spin-time 5
```

```
# Ctrl+C in T1 closes Isaac Sim
```

Observations (recorded)

Quantity	Value
`/odom` after 0.2 m/s for 4 simulated s	moved 0.82 m (`drive_sim.py`, window mode) and 0.86 m (automated test, headless); heading changed -0.006 rad
`/odom` after 0.5 rad/s for 3 simulated s	turned 1.37 rad (`drive_sim.py`) and 1.51 rad (test), moved 0.01 m
TF	`odom -> base_footprint` plus the transforms to every URDF link (`base_link`, wheels, caster, `lidar_link`)
Same drives with wall-clock `isaac-drive` (window mode)	1.32 m and 2.2 rad: the simulation ran about 1.5 x real time

Verification (starts its own headless Isaac Sim, 3-6 min; close T1 first)

```
pixi run test-isaac-p1
```

11 checks: `/clock`, `/odom`, TF to every link and `/joint_states` exist; the two drives move the robot 0.6-1.0 m and turn it 1.1-1.9 rad. Recorded: 11/11.

Questions

1. Why is the distance not exactly 0.80 m here, while a kinematic simulator would be exact? Name two effects a physics engine adds.
2. Why is `ROS_DOMAIN_ID` the same (78) in both programs, and why different from the other labs (default)?
3. Which of the three URDF additions would you remove first to see the robot tip over, and what do you expect?
4. How would the robot behave on a slippery floor? Which part of the graph would you change?
5. Why does the distance driven by `isaac-drive` depend on how fast the simulation runs, while `drive_sim.py` stays close to the ideal value? `r` n6. *(Extension)* Make the robot drive in a circle of radius 0.5 m: which `v` and `w`?

If it does not work

Symptom	Fix
"The filename or extension is too long"	the install is not in `C:\ix`; run `pixi run isaac-install`
Window opens but is black	wait; the first frames need shader compilation
`isaac-ros topic list` shows nothing	Isaac Sim is still starting (wait for `project 1 ready`)
Crash at start-up on a new NVIDIA driver	try the tested driver or run without `--gui`
`drive_sim.py` says "no /clock"	T1 not ready (wait for `project 1 ready`)

Lab I2 - LiDAR in Isaac Sim, steered by a ready-made obstacle avoider

Aim. Give the Isaac Sim robot a 360 degree lidar that publishes a standard `sensor_msgs/LaserScan`, then drive it with the obstacle-avoider node that is part of this folder (`ws\src\diffbot_navigation`), which was written for a different, kinematic simulator and is used **unmodified**. The point: ROS 2 interfaces let you swap the simulator behind them.

Time. 30 min. **Needs.** Only this folder, an **NVIDIA RTX GPU** (tested: RTX 5050 Laptop, 8 GB, driver 592.82) and about 15 GB of disk for Isaac Sim, which `pixi run isaac-install` downloads once for all Isaac labs on the machine. By running the scripts you accept NVIDIA's Isaac Sim licence.

This folder is a complete project

Everything this lab needs is in this folder. Nothing from the other labs is required.

Requirements: Windows 10/11, PowerShell, pixi (no administrator rights needed; install steps are in README-SETUP.md), and an NVIDIA RTX GPU with a current driver; about 15 GB disk for Isaac Sim (shared by all Isaac labs, installed once into `C:\ix`); internet for the first install.

In this folder	
`isaac\`	the Isaac Sim scene scripts and ROS 2 nodes
`ws\src\`	the ROS 2 packages; `ws\install\` appears after `pixi run build`
`scripts\`	helper scripts and the automated tests
`pixi.toml`	the environment and every `pixi run ...` command; `README.md` this manual; `tools\replay.ps1` re-runs the manual's commands as a self-test

Set it up once. `pixi install` puts ROS 2 Jazzy into `C:\rlabs` (a few GB; pixi's package cache makes the second lab on the same machine fast). Open PowerShell in this folder and run:

```
pixi install
pixi run verify
pixi run build
```

(`pixi run build` compiles this lab's packages into `ws\install`; run it again after every edit in `ws\src`.)

Run one lab at a time, and everything from this folder.

Theory

The scene is an 8 m x 6 m room (robot at the origin looking along +x) plus two 0.6 m boxes at (1.5, 1.0) and (-1.5, -1.0). A PhysX ray-casting lidar on `lidar_link` (8 cm ahead of the robot centre) has 720 rays (0.5 degree steps), range 0.12-6 m, and is published on `/scan`.

	the kinematic simulator the avoider was written for	this lab
who moves the robot	kinematic Python simulator	Isaac Sim physics
`/scan`	360 rays, 1 degree	**720 rays, 0.5 degree**
`/cmd_vel` consumer	`diffbot_sim`	OmniGraph differential controller
the avoider	`obstacle_avoider`	**the same node, no change** (it reads `angle_min` and `angle_increment` from every message, so the ray count does not matter)

Expected scan at the start, from the geometry: straight ahead 4.00 - 0.08 = **3.92 m**, left and right **3.00 m**, behind 4.00 + 0.08 = **4.08 m** (a live window-mode read gave 3.90 / 3.00 / 4.10 because the robot had already drifted a few centimetres).

Procedure

1. T1 - Isaac Sim with the lidar (a room with two boxes).

```
pixi run isaac-p2 --gui
```

2. T2 - check the scan against the room. Index 360 is straight ahead, 540 left (+90 deg), 180 right (-90 deg), 0 behind (+-180 deg).

```
pixi run isaac-ros topic echo /scan --no-daemon --once --spin-time 5 --field angle_increment
$o = pixi run isaac-ros topic echo /scan --no-daemon --once --spin-time 5 --field ranges | Out-
String
$r = [regex]::Match($o, '\[(.*?)\]').Groups[1].Value -split ',\s*'
"rays: $($r.Count)"
"front {0:N2}  left {1:N2}  back {2:N2}  right {3:N2}" -f [double]$r[360],[double]$r[540],
[double]$r[0],[double]$r[180]
```

3. T2 - rate of the scan (messages per wall-clock second; stop it with Ctrl+C - here it stops itself after 30 s).

```
pixi run isaac-ros topic hz /scan --spin-time 5
```

4. T2 - start the obstacle avoider, unchanged. (Build it once first with `pixi run build`.) The robot starts to wander and turns away from walls and boxes.

```
pixi run isaac-avoider
```

5. T3 - is it avoiding? Position of the robot five times, 8 s apart (it should move through the room, never stop at a wall):

```
1..5 | ForEach-Object { pixi run isaac-ros topic echo /odom --no-daemon --once --spin-time 5 --
field pose.pose.position; Start-Sleep 8 }
```

```
# Ctrl+C in T1 and T2
```

Observations (recorded)

Quantity	Value
Scan at the start	720 rays, 0.5 degree steps (`angle_increment` 0.008727); front / left / back / right = 3.92 / 3.00 / 4.08 / 3.00 m; about 33 messages per wall-clock second
The avoider in Isaac Sim	12.3-12.5 m driven in 60 simulated seconds; closest approach of the robot centre to any wall or box 0.46-0.58 m (robot radius about 0.17 m)

Verification (own headless Isaac Sim, 4-6 min; close T1 and T2 first)

```
pixi run test-isaac-p2
```

9 checks: Isaac Sim publishes `/clock`, `/odom`, `/scan`; the scan is a full circle in frame `lidar_link`; the ranges at 0 / 90 / 180 / -90 degrees match the room to 12 cm; the unchanged avoider drives the robot more than 4 m in 60 simulated seconds, its centre never comes closer to a wall or box than the robot radius (0.17 m), and it stays inside the room.

Questions

1. Why did the avoider need no change although `/scan` has twice as many rays? Which two fields of `LaserScan` make that work?
2. The measured front range is 3.92 m, not 4.00 m. Why?
3. A kinematic simulator can count collisions directly. How does this lab measure "did not hit" instead?
4. What would break if Isaac Sim published `/scan` in frame `base_link` instead of `lidar_link`?
5. *(Extension)* Add a third box in `isaac\scripts\p2_lidar.py` (`boxes=[...]`) and check the avoider still copes.

If it does not work

Symptom	Fix
`/scan` missing	wait for `project 2 ready`; `isaac-ros topic list --no-daemon --spin-time 5`
The robot does not move	`pixi run isaac-avoider` runs on domain 78 automatically; T1 must be Isaac project 2, not another project
The range line prints an error about `[double]`	the echo printed nothing: wait for `project 2 ready` and repeat

Lab I3 - Camera in Isaac Sim: steer towards a red cube by its colour

Aim. Add an RGB + depth camera to the Isaac Sim robot and run a small ROS 2 node that finds a red cube in the image, turns the robot towards it, drives up to it and stops 0.55 m in front, using the depth image to measure the distance. A green cube is a decoy.

Time. 45 min. **Needs.** Only this folder, an **NVIDIA RTX GPU** (tested: RTX 5050 Laptop, 8 GB, driver 592.82) and about 15 GB of disk for Isaac Sim, which `pixi run isaac-install` downloads once for all Isaac labs on the machine. By running the scripts you accept NVIDIA's Isaac Sim licence.

This folder is a complete project

Everything this lab needs is in this folder. Nothing from the other labs is required.

Requirements: Windows 10/11, PowerShell, pixi (no administrator rights needed; install steps are in README-SETUP.md), and an NVIDIA RTX GPU with a current driver; about 15 GB disk for Isaac Sim (shared by all Isaac labs, installed once into `C:\ix`); internet for the first install.

In this folder	
`isaac\`	the Isaac Sim scene scripts and ROS 2 nodes
`scripts\`	helper scripts and the automated tests
`pixi.toml`	the environment and every `pixi run ...` command; `README.md` this manual; `tools\replay.ps1` re-runs the manual's commands as a self-test

Set it up once. `pixi install` puts ROS 2 Jazzy into `C:\rlabs` (a few GB; pixi's package cache makes the second lab on the same machine fast). Open PowerShell in this folder and run:

```
pixi install
pixi run verify
```

Run one lab at a time, and everything from this folder.

Theory

- Camera on the front of the body, 320 x 240 pixels, 90 degree field of view. Topics: `/camera/image_raw` (`sensor_msgs/Image`, `rgb8`: 3 bytes per pixel) and `/camera/depth` (`32FC1`: one float, metres, per pixel).
- The ROS 2 node `isaac\ros_nodes\color_follower.py` (about 60 lines, numpy only) does, per image: 1. a pixel is **red** if `R > 120`, `R - G > 70`, `R - B > 70`; 2. `error = (mean column of red pixels - centre) / half-width`, in -1 ... +1 (negative = target on the left); 3. turn `w = -1.2 * error`; drive forward only when the target is within 12 % of the centre; 4. stop when the depth in the middle of the picture is 0.55 m or less; with no red in view, turn slowly to search.
- Scene: the red cube is at (2.2, 1.3) - about 31 degrees to the left of the robot's nose - and the green decoy at (2.2, -1.2).

Procedure

1. T1 - Isaac Sim with the camera (window: robot, red cube, green cube).

```
pixi run isaac-p3 --gui
```

2. T2 - the images exist. Encoding, size and rate (the rate is in simulated time).

```
pixi run isaac-ros topic list --no-daemon --spin-time 5
pixi run isaac-ros topic echo /camera/image_raw --no-daemon --once --spin-time 5 --field encoding
pixi run isaac-ros topic echo /camera/depth --no-daemon --once --spin-time 5 --field encoding
pixi run isaac-ros topic echo /camera/image_raw --no-daemon --once --spin-time 5 --field width
```

3. T2 - start the controller. The robot turns left until the red cube is in the middle, then drives to it and stops. The terminal prints what the node sees.

```
pixi run isaac-follower
```

4. T3 - did it work? Wait about 40 s, then read the final position. The cube is at (2.2, 1.3); the robot should stop 0.55 m + half its length in front of it.

```
Start-Sleep 45
pixi run isaac-ros topic echo /odom --no-daemon --once --spin-time 5 --field pose.pose.position
```

Compute the distance from (x, y) to (2.2, 1.3). Recorded: 0.77 m (the node stops when the camera is 0.55 m from the cube's front face).

```
# Ctrl+C in T1 and T2
```

Observations (recorded)

Quantity	Value
Images	320 x 240, `rgb8` and `32FC1`
Start	red cube 31 degrees to the left, green decoy on the right
End	robot 0.77 m from the cube centre, 1 degree off the nose; it did not go for the decoy
Depth straight ahead (cube at 0.50 m by geometry)	0.48 m

Verification (own headless Isaac Sim, 4-6 min; close T1/T2 first)

```
pixi run test-isaac-p3
```

12 checks: image sizes and encodings; the red pixels are in the left part of the first picture; the depth value matches the geometry; the robot ends nearer to the red cube than to the green decoy, close to it and facing it.

Questions

1. Why is `R > 120` alone not a good "red" test? What does `R - G > 70` and `R - B > 70` add?

2. What would the node do if the red cube were behind the robot? If there were two red cubes?
3. Why does the node use *depth* to stop instead of the size of the red blob? Which one would a cheap robot with only an RGB camera have to use?
4. Change the colour test to follow the green cube instead. Which constants change?
5. *(Extension)* Add a lidar (see the LiDAR lab) and approach the cube without hitting a box.

If it does not work

Symptom	Fix
Robot only spins	the cube is not in view; check the camera picture in the Isaac Sim window; the colour test needs the light the script adds
`isaac-follower` prints nothing	T1 is not at `project 3 ready`
Camera images black	run with `--gui` once to let the shaders compile

Lab I4 - Waypoints in Isaac Sim: red, green, blue

Aim. Write down a mission as three points and let a small ROS 2 node drive DiffBot to them in order, using only `/odom` for position. Three coloured squares painted on the floor of the Isaac Sim room mark the points.

Time. 30 min. **Needs.** Only this folder, an **NVIDIA RTX GPU** (tested: RTX 5050 Laptop, 8 GB, driver 592.82) and about 15 GB of disk for Isaac Sim, which `pixi run isaac-install` downloads once for all Isaac labs on the machine. By running the scripts you accept NVIDIA's Isaac Sim licence.

This folder is a complete project

Everything this lab needs is in this folder. Nothing from the other labs is required.

Requirements: Windows 10/11, PowerShell, pixi (no administrator rights needed; install steps are in README-SETUP.md), and an NVIDIA RTX GPU with a current driver; about 15 GB disk for Isaac Sim (shared by all Isaac labs, installed once into `C:\ix`); internet for the first install.

In this folder	
`isaac\`	the Isaac Sim scene scripts and ROS 2 nodes
`scripts\`	helper scripts and the automated tests
`pixi.toml`	the environment and every `pixi run ...` command; `README.md` this manual; `tools\replay.ps1` re-runs the manual's commands as a self-test

Set it up once. `pixi install` puts ROS 2 Jazzy into `C:\rlabs` (a few GB; pixi's package cache makes the second lab on the same machine fast). Open PowerShell in this folder and run:

```
pixi install
pixi run verify
```

Run one lab at a time, and everything from this folder.

Theory

Positions are in the `odom` frame; the robot starts at (0, 0) looking along +x:

Waypoint	Square	(x, y) in m
A	red	(1.5, 0.0)
B	green	(1.5, 1.5)
C	blue	(-0.5, 1.5)

The squares are flat boxes **without collision** - paint, not obstacles. The node `isaac\ros_nodes\waypoints.py` is a "go to a point" controller in 60 lines. For the current goal, every time `/odom` arrives:

```
heading_error = direction_to_goal - my_heading     (wrapped to -180 ... +180 degrees)
turn  w = 1.5 * heading_error   (limited to 1.0 rad/s)
drive v = only when |heading_error| < 0.5 rad, slower when close (at most 0.25 m/s)
closer than 0.12 m -> "arrived", next goal; after C: stop
```

`my_heading = 2 * atan2(z, w)` from the odometry quaternion. The coordinates are written twice (scene script and node): change both to move a square.

Procedure

1. T1 - Isaac Sim (window: room with the three squares).

```
pixi run isaac-p4 --gui
```

2. T2 - start the mission. The node prints `arrived at ...` for each point and `all waypoints visited: stopping`. (6-26 simulated seconds per leg; allow about a minute.)

```
pixi run isaac-waypoints
```

3. T3 - where did it stop? It should stand on the blue square, (-0.5, 1.5), within about 10 cm.

```
pixi run isaac-ros topic echo /odom --no-daemon --once --spin-time 5 --field pose.pose.position
```

```
# Ctrl+C in T1 and T2
```

Observations (recorded)

Quantity	Value
Arrival at A, B, C	about 6-9, 14-17 and 23-26 simulated seconds (three runs)
Final position	10 cm from the centre of C; moved 0.1 cm in the last 2 s (stopped)
Room	never left the 8 x 6 m room

Verification (own headless Isaac Sim, 3-5 min; close T1/T2 first)

```
pixi run test-isaac-p4
```

7 checks, judged from `/odom` only: the robot starts at the origin; it comes within 20 cm of A, B and C **in that order**; it stops after C (moves less than 3 cm in the last 2 s) standing on C; it never leaves the room.

Questions

1. Why does the node turn first and drive only when it faces the goal within 0.5 rad? What happens if it always drives?
2. What changes if `TOLERANCE` is 0.02 m? (Think of a proportional controller and wheel slip.)
3. A full navigator also plans around obstacles and reports failure. What does this node not have? When would that matter?
4. Add a fourth square D at (-0.5, 0.0) and a loop A -> B -> C -> D -> A. Which two files do you edit?
5. *(Extension)* Put a box between B and C and add a lidar to get round it.

If it does not work

Symptom	Fix
Node says "waiting for /odom"	T1 is not ready

Symptom	Fix
The mission does not repeat	the node stops after C; restart T2 (and T1, which resets the robot to the origin)

Lab I5 - Line follower with a downward camera

Aim. A camera looks down at the floor; a ROS 2 node finds the dark pixels of a black line and steers to keep them in the middle of the picture. The robot follows a line with a smooth 0.6 m bend to the left and stops where the line ends.

Time. 30 min. **Needs.** Only this folder, an **NVIDIA RTX GPU** (tested: RTX 5050 Laptop, 8 GB, driver 592.82) and about 15 GB of disk for Isaac Sim, which `pixi run isaac-install` downloads once for all Isaac labs on the machine. By running the scripts you accept NVIDIA's Isaac Sim licence.

This folder is a complete project

Everything this lab needs is in this folder. Nothing from the other labs is required.

Requirements: Windows 10/11, PowerShell, pixi (no administrator rights needed; install steps are in README-SETUP.md), and an NVIDIA RTX GPU with a current driver; about 15 GB disk for Isaac Sim (shared by all Isaac labs, installed once into `C:\ix`); internet for the first install.

In this folder	
`isaac\`	the Isaac Sim scene scripts and ROS 2 nodes
`scripts\`	helper scripts and the automated tests
`pixi.toml`	the environment and every `pixi run ...` command; `README.md` this manual; `tools\replay.ps1` re-runs the manual's commands as a self-test

Set it up once. `pixi install` puts ROS 2 Jazzy into `C:\rlabs` (a few GB; pixi's package cache makes the second lab on the same machine fast). Open PowerShell in this folder and run:

```
pixi install
pixi run verify
```

Run one lab at a time, and everything from this folder.

Theory

- Camera: 160 x 120 pixels, 90 degree field of view, on the front edge of the body, 21 cm above the floor, tilted **50 degrees down**. It sees the floor from about 1 cm to 0.9 m ahead. (It must be well in front of the roof, otherwise its steepest rays see the robot's own dark body.)
- Scene: a pale mat with a black line - straight, a smooth 0.6 m bend to the left, straight again.
- `isaac\ros_nodes\line_follower.py`: 1. a pixel is **dark** if its grey value is below 70 % of the picture's **median** grey value (the "black" line is rendered mid-grey, about 100 of 255, because the light reflects off it, so a fixed threshold fails); 2. `error = (mean column of the dark pixels - centre) / (width / 2)`, -1 ... +1, negative = line on the left; 3. turn `w = -1.6 * error`; drive at 0.12 m/s, slower in sharp turns; 4. fewer than 30 dark pixels = line lost = **stop** (this is how it ends at the end of the line).
- This is a **proportional controller** on the measured error, closed through the camera and the physics.

Procedure

1. T1 - Isaac Sim (window: mat, black line, robot standing on its start).

```
pixi run isaac-p5 --gui
```

2. T2 - the image. Size and encoding; in the Isaac Sim window you see the same picture the node uses.

```
pixi run isaac-ros topic echo /camera/image_raw --no-daemon --once --spin-time 5 --field encoding
pixi run isaac-ros topic echo /camera/image_raw --no-daemon --once --spin-time 5 --field width
```

3. T2 - follow the line. The node prints its error and speeds, and `line lost: stopped` at the end (about 55 simulated seconds).

```
pixi run isaac-line
```

4. T3 - where did it end? The line ends at x = 3.5 m, after the bend (y = 0.60 m). The robot should be near there and standing still.

```
pixi run isaac-ros topic echo /odom --no-daemon --once --spin-time 5 --field pose.pose.position
```

```
# Ctrl+C in T1 and T2
```

Observations (recorded)

Quantity	Value
Image	160 x 120 `rgb8`; 1718 dark pixels at the start (the line), floor median 243 of 255
Run	followed the line to its end in about 55 simulated seconds; ended at y = 0.60 m after the bend, then stopped
Accuracy	0.4 cm from the line on average, 1.6 cm at worst (measured from `/odom` against the line's formula)

Verification (own headless Isaac Sim, 4-6 min; close T1/T2 first)

```
pixi run test-isaac-p5
```

11 checks: image size and encoding; the line is visible near the middle of the first picture (150-3500 dark pixels, none from the robot's own body); the floor is pale (median > 150); the follower's rule says "drive on, hardly turn"; the robot reaches x > 3.0 m, after the bend (y within 15 cm of 0.6 m); mean distance from the line < 6 cm and worst < 18 cm; it stops at the end.

Questions

1. Why is "darker than 70 % of the median" better than "grey value below 70"? What would go wrong with a black floor?
2. What happens with `w = -3.0 * error`? With `-0.5 * error`? Predict, then change the constant in `line_follower.py` and run again.
3. The camera looks down at 50 degrees, not straight down. What does the robot gain, what does it lose?
4. A proportional controller cannot remove a constant offset in a curve. How would a PID controller help?
5. *(Extension)* Make the line a closed loop (`isaac\scripts\line_track.py`) or stop at a red patch (a colour test like "R > 120, R - G > 70, R - B > 70").

If it does not work

Symptom	Fix
Robot stops immediately	fewer than 30 dark pixels: the camera does not see the line; restart T1
Half of the picture counts as "dark"	the camera sees the robot's own body: check `position` / `pitch_deg` of `add_camera`

Lab I6 - Climb a ramp, judged by the IMU

Aim. Put an **IMU** on the robot and climb a 10 degree ramp onto a flat deck. A ROS 2 node reads only the IMU's tilt to decide "I am on the ramp" and "I am on the deck, stop". This is something the kinematic simulator of Labs 3-5 cannot do - it has no tilt - but a physics engine can.

Time. 30 min. **Needs.** Only this folder, an **NVIDIA RTX GPU** (tested: RTX 5050 Laptop, 8 GB, driver 592.82) and about 15 GB of disk for Isaac Sim, which `pixi run isaac-install` downloads once for all Isaac labs on the machine. By running the scripts you accept NVIDIA's Isaac Sim licence.

This folder is a complete project

Everything this lab needs is in this folder. Nothing from the other labs is required.

Requirements: Windows 10/11, PowerShell, pixi (no administrator rights needed; install steps are in README-SETUP.md), and an NVIDIA RTX GPU with a current driver; about 15 GB disk for Isaac Sim (shared by all Isaac labs, installed once into `C:\ix`); internet for the first install.

In this folder	
`isaac\`	the Isaac Sim scene scripts and ROS 2 nodes
`scripts\`	helper scripts and the automated tests
`pixi.toml`	the environment and every `pixi run ...` command; `README.md` this manual; `tools\replay.ps1` re-runs the manual's commands as a self-test

Set it up once. `pixi install` puts ROS 2 Jazzy into `C:\rlabs` (a few GB; pixi's package cache makes the second lab on the same machine fast). Open PowerShell in this folder and run:

```
pixi install
pixi run verify
```

Run one lab at a time, and everything from this folder.

Theory

An **IMU** (inertial measurement unit, the sensor in every phone) reports `sensor_msgs/Imu`:

Field	Meaning	Standing still on level ground
`angular_velocity`	how fast the body turns, rad/s	0, 0, 0
`linear_acceleration`	the acceleration it **feels**, gravity included, m/s^2	**(0, 0, +9.81)**
`orientation`	tilt as a quaternion	no tilt

Scene: a ramp starting at x = 0.8 m, 2.0 m long, rising at 10 degrees (so the deck is 2.0 x tan 10 deg = **0.353 m** high), then a flat 1 m deck. The robot starts at the origin looking up the ramp.

`isaac\ros_nodes\ramp_climber.py` converts the orientation to a **slope** (`slope = -pitch`; in ROS a positive pitch tips the nose *down*, so nose-up is negative pitch) and runs three states:

```
waiting  --slope > 5 deg-->  climbing  --slope < 2 deg for 1 s-->  on the deck: stop
any state --slope > 25 deg--> too steep: stop
```

The one-second timer uses the IMU's own (simulated) time stamps, never the wall clock. Isaac Sim's IMU here is **noise-free**; a real one is noisy and drifts, which is why real robots filter it.

Procedure

1. T1 - Isaac Sim (window: ramp and deck).

```
pixi run isaac-p6 --gui
```

2. T2 - read the IMU at rest.

```
pixi run isaac-ros topic echo /imu --no-daemon --once --spin-time 5
```

3. T2 - climb. The node prints `on the ramp: slope 10.0 degrees` and, at the top, `on the deck: level again`, then stops the robot.

```
pixi run isaac-ramp
```

4. T3 - confirm with the odometry. The robot should be on the deck: `z` about 0.35 m above its start, x between 2.8 m and 3.8 m, and stand still. (If the node has not printed `on the deck: level again` yet, repeat the command after a few seconds: the numbers above are only valid once it has.)

```
pixi run isaac-ros topic echo /odom --no-daemon --once --spin-time 5 --field pose.pose.position
```

```
# Ctrl+C in T1 and T2
```

Observations (recorded)

Quantity	Value
At rest	frame `base_link`, acceleration (0.00, 0.00, 9.81), tilt 0.00 degrees
On the ramp	steepest slope 10.0 degrees (the ramp has 10)
Climb in `/odom`	0.35 m (deck: 0.353 m)
End	on the deck, slope 0.0 degrees, stopped before the end of the deck

Verification (own headless Isaac Sim, 4-6 min; close T1/T2 first)

```
pixi run test-isaac-p6
```

11 checks: `/imu` frame is `base_link`; at rest |a| = 9.81 +- 0.8 and no tilt; on the ramp the IMU measures 10 +- 4 degrees; the robot is on the deck (x > 2.85 m, climbed 0.35 +- 0.1 m); level again on the deck; stopped there (moved under 3 cm in the last 2 s) before the end of the deck. Recorded: 11/11.

Questions

1. Why does the acceleration read +9.81 on the z axis when the robot stands still, not 0?
2. In ROS a positive pitch tips the nose *down*. How does the node turn the quaternion into a "nose-up is positive" slope?
3. Why does the node wait 1 s of "level" before deciding it is on the deck, instead of stopping the moment the slope drops below 2 degrees?
4. Make the ramp 20 degrees (`RAMP_ANGLE` in `p6_ramp.py` **and** in the test) and see where the robot slips or tips. Which state does the node end in?
5. *(Extension)* Add noise to the IMU reading in your node and filter it with a moving average; compare the climb detection with and without it.

If it does not work

Symptom	Fix
`/imu` missing	T1 not ready (`project 6 ready`)
The node prints `too steep`	the slope exceeded 25 degrees: the robot tipped; restart T1

Lab U1 - "ROS 2 from Scratch" (Packt), chapter 9: Python and C++ nodes

Aim. Build and run the *unmodified* chapter-9 code of the Packt book **ROS 2 from Scratch**: publishers, a subscriber, a service, parameters, namespaces, launch files and an action - each in Python and in C++.

Time. 90 min. **Needs.** Only this folder, plus a C++ compiler (Visual Studio Build Tools, C++ workload) for the book's C++ nodes and its interface package. The book's repository is downloaded by `pixi run build-packt`. The code lives in `upstream\ros2-from-scratch\ch9` and is built into its own overlay `upstream_ws\packt`; the clone is never modified.

This folder is a complete project

Everything this lab needs is in this folder. Nothing from the other labs is required.

Requirements: Windows 10/11, PowerShell, pixi (no administrator rights needed; install steps are in README-SETUP.md), and Visual Studio Build Tools (C++ workload); internet access once, to download the book's repository from GitHub.

In this folder	
`upstream\`	the original repository (downloaded by the first build task, pinned commit); `upstream_ws\` its build
`scripts\`	helper scripts and the automated tests
`pixi.toml`	the environment and every `pixi run ...` command; `README.md` this manual; `tools\replay.ps1` re-runs the manual's commands as a self-test

Set it up once. `pixi install` puts ROS 2 Jazzy into `C:\rlabs` (a few GB; pixi's package cache makes the second lab on the same machine fast). Open PowerShell in this folder and run:

```
pixi install
pixi run verify
```

Run one lab at a time, and everything from this folder.

Theory

Package	Content
`my_py_pkg`, `my_cpp_pkg`	the same nodes in two languages: `number_publisher`, `number_counter` (service `reset_counter`), `count_until_server` / `count_until_client` (action)
`my_robot_interfaces`	`ResetCounter.srv` (`int64 reset_value -> bool success, string message`), `ActivateTurtle.srv`, `CountUntil.action` (goal `target_number, delay`; feedback `current_number`; result `reached_number`)
`turtle_controller` (Python), `turtle_controller_cpp`	drives a turtle in a circle, changes the pen colour, service `activate_turtle`
`my_robot_bringup`	launch files and YAML parameter files

A **namespace** (`/abc` , `/t1`) puts every name of a node under a prefix, so the same node can run twice. A **remap** renames a topic at start-up.

Procedure

Commands that need the book's own interface types go through its overlay. This one-line shortcut (re-type it in every new terminal) makes them short:

```
function packt { pixi run cmd /c scripts\in_upstream.bat packt @args }
```

1. Build the book's six packages (about 1-5 minutes the first time, seconds afterwards).

```
pixi run build-packt
```

Part A - publishers, counter, service, parameters

2. T1 - start the number app. Two Python publishers (inline parameters / YAML parameters) feed one C++ counter.

```
pixi run demo-packt-numbers
```

3. T2 - inspect. The launch file asks for a remap `/number -> /my_number` , but the topic really is `/abc/number` (see question 1).

```
pixi run ros node list --no-daemon --spin-time 5
pixi run ros topic list --no-daemon --spin-time 5
pixi run ros topic echo /abc/number --no-daemon --once --spin-time 5
```

4. T2 - parameters. `num_pub1` was configured in the launch file (period 1.5 s), `num_pub2` from the YAML file (number 4). Changing `number` at run time takes effect immediately (a callback in the node).

```
pixi run ros param get /abc/num_pub1 publish_period --no-daemon --spin-time 5
pixi run ros param get /abc/num_pub2 number --no-daemon --spin-time 5
pixi run ros param set /abc/num_pub1 number 10 --spin-time 5
```

5. T2 - call the C++ service. It must refuse a negative value and a value above the current count, and accept a valid one (watch T1: `Reset counter to 0`).

```
function packt { pixi run cmd /c scripts\in_upstream.bat packt @args }
packt ros2 service call /abc/reset_counter my_robot_interfaces/srv/ResetCounter "{reset_value: -5}"
packt ros2 service call /abc/reset_counter my_robot_interfaces/srv/ResetCounter "{reset_value:
99999999}"
packt ros2 service call /abc/reset_counter my_robot_interfaces/srv/ResetCounter "{reset_value: 0}"
```

```
# Ctrl+C in T1
```

Part B - two turtles, two namespaces

6. T1 - two turtlesim windows (red and green background), each with its own controller (`turtle_velocity` 1.5 and 0.5 from the YAML file).

```
pixi run demo-packt-turtles
```

7. T2 - compare the two controllers, then stop one.

```
function packt { pixi run cmd /c scripts\in_upstream.bat packt @args }
pixi run ros param get /t1/turtle_controller turtle_velocity --no-daemon --spin-time 5
pixi run ros param get /t2/turtle_controller turtle_velocity --no-daemon --spin-time 5
packt ros2 service call /t1/activate_turtle my_robot_interfaces/srv/ActivateTurtle "{activate:
false}"
```

Watch the windows: the left turtle stops, the other keeps circling. Confirm with the topic (no message arrives for `/t1`, messages keep coming for `/t2`; `linear.x` is 0.5, or 1.0 while the turtle is in the right half of the window, because the controller doubles the speed there):

```
pixi run ros topic echo /t2/turtle1/cmd_vel --no-daemon --once --spin-time 5
```

```
pixi run ros topic echo /t1/turtle1/cmd_vel --no-daemon --spin-time 5
```

```
# Ctrl+C in T1
```

Part C - the action `count_until`, Python and C++

8. T1 - action server (Python), T2 - client. The client sends goal "count to 5, 0.5 s apart", prints feedback 1..5 and the result. **The client keeps running after the result: press Ctrl+C.**

```
pixi run cmd /c scripts\in_upstream.bat packt ros2 run my_py_pkg count_until_server
```

```
pixi run cmd /c scripts\in_upstream.bat packt ros2 run my_py_pkg count_until_client
```

Repeat with the C++ pair (stop the Python server first): the behaviour is the same.

```
# Ctrl+C in T1 (server) and T2 (client)
```

```
pixi run cmd /c scripts\in_upstream.bat packt ros2 run my_cpp_pkg count_until_server
```

```
pixi run cmd /c scripts\in_upstream.bat packt ros2 run my_cpp_pkg count_until_client
```

```
# Ctrl+C in T1 (server) and T2 (client)
```

Observations (recorded)

Item	Value
Topic of the number app	`/abc/number` (not `/my_number`)

Item	Value
`reset_counter` replies	`-5 -> success=False, 'Cannot reset counter to a negative value'`; too large `-> success=False, 'Reset value must be lower than current counter value'`; `0 -> success=True`
`turtle_velocity`	`/t1`: 1.5, `/t2`: 0.5; `cmd_vel` linear.x 1.5 and 0.5
`activate_turtle(false)` on `/t1`	reply `Stopping the turtle`; `/t1/turtle1/cmd_vel` falls silent, `/t2` unaffected
`count_until`	feedback 1,2,3,4,5 at 0.5 s steps, result 5 - identical in Python and C++

Verification

```
pixi run test-packt
```

25 checks against the book's code: C++ service (negative / too high / valid), YAML and inline launch parameters, the fast turtle travels about 22 units in 10 s against 8-9 for the slow one, `activate_turtle`, and the `count_until` action in both languages (accept, 5 feedback messages, result, reject a negative target, cancel a running goal).

Questions

1. In `number_app.launch.py` the remap is `("/number", "/my_number")`, yet the topic is `/abc/number`. Why does the rule not match, and how would you write a remap that works?
2. What does `ros2 param set /abc/num_pub1 number 10` change in the counter's total, and why does it work while the node is running?
3. Why does the service refuse `reset_value` above the current count? Where in `number_counter.cpp` is that decided?
4. What is the advantage of an action over a service for `count_until`? What would `Ctrl+C` in the client do to a *running* goal?
5. *(Extension)* Start the second Python publisher with another namespace and see that the counter is not affected.

If it does not work

Symptom	Fix
`in_upstream.bat: upstream_ws\packt is not built`	`pixi run build-packt`
Service call hangs on "waiting for service"	T1 is not running, or the namespace is wrong (`/abc/reset_counter`)
Client sees no server right after you restarted the server	wait 5 s; ROS still remembers the old server for a moment

Lab U2 - turtlesim built from source (`ros_tutorials`)

Aim. Compile ROS's own teaching simulator **turtlesim** from its source repository (C++, Qt5) and drive it through every interface type: topic, services, an action, and the `mimic` node that links two simulators.

Time. 45 min. **Needs.** Only this folder, plus a C++ compiler (Visual Studio Build Tools, C++ workload). The repository is downloaded by `pixi run build-turtlesim`. Source: `upstream\ros_tutorials-jazzy` (branch `jazzy`), built into `upstream_ws\turtlesim`; the pixi package of the same name is shadowed by it.

This folder is a complete project

Everything this lab needs is in this folder. Nothing from the other labs is required.

Requirements: Windows 10/11, PowerShell, pixi (no administrator rights needed; install steps are in README-SETUP.md), and Visual Studio Build Tools (C++ workload); internet access once, to download `ros/ros_tutorials` from GitHub.

In this folder	
`upstream\`	the original repository (downloaded by the first build task, pinned commit); `upstream_ws\` its build
`scripts\`	helper scripts and the automated tests
`pixi.toml`	the environment and every `pixi run ...` command; `README.md` this manual; `tools\replay.ps1` re-runs the manual's commands as a self-test

Set it up once. `pixi install` puts ROS 2 Jazzy into `C:\rlabs` (a few GB; pixi's package cache makes the second lab on the same machine fast). Open PowerShell in this folder and run:

```
pixi install
pixi run verify
```

Run one lab at a time, and everything from this folder.

Theory

turtlesim draws one turtle on an 11 x 11 canvas.

Interface	Name	Type
topic in	`/turtle1/cmd_vel`	`geometry_msgs/Twist` (linear.x forward, angular.z turn)
topic out	`/turtle1/pose`	`turtlesim/Pose` (x, y, theta, velocities)
services	`/spawn`, `/kill`, `/clear`, `/reset`, `/turtle1/teleport_absolute`, `/turtle1/teleport_relative`, `/turtle1/set_pen`	`turtlesim/srv/...`
action	`/turtle1/rotate_absolute`	`turtlesim/action/RotateAbsolute` (goal `theta`; feedback `remaining`; result `delta`)

Two running simulators must differ in their **namespace**, otherwise their topics collide.

Procedure

Shortcut to run `ros2` with the source-built overlay (re-type it in every new terminal):

```
function ts { pixi run cmd /c scripts\in_upstream.bat turtlesim @args }
```

1. Build turtlesim from source (about 40 s; the first time it compiles C++ and Qt).

```
pixi run build-turtlesim
```

2. T1 - start it (a blue window with one turtle in the middle).

```
pixi run demo-turtlesim
```

3. T2 - prove it is *your* build, then read the pose. The first path must point into `upstream_ws\turtlesim`, the second (without the overlay) into the pixi environment (a folder under `C:\rlabs`).

```
function ts { pixi run cmd /c scripts\in_upstream.bat turtlesim @args }
ts ros2 pkg prefix turtlesim
pixi run ros pkg prefix turtlesim
ts ros2 topic echo /turtle1/pose --no-daemon --spin-time 5 --once
```

4. T2 - topic: move the turtle. One `Twist` message: 2 units/s forward, 1.8 rad/s turn, for one second (turtlesim stops the turtle after 1 s without commands).

```
function ts { pixi run cmd /c scripts\in_upstream.bat turtlesim @args }
ts ros2 topic pub --once /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0}, angular: {z:
1.8}}"
```

5. T2 - services. A second turtle, a jump, a red thick pen, a clean canvas, and removal of the second turtle. (`'off': 0` is quoted because YAML would read a bare `off` as a boolean.)

```
function ts { pixi run cmd /c scripts\in_upstream.bat turtlesim @args }
ts ros2 service call /spawn turtlesim/srv/Spawn "{x: 2.0, y: 2.0, theta: 0.0, name: turtle2}"
ts ros2 service call /turtle1/teleport_absolute turtlesim/srv/TeleportAbsolute "{x: 8.0, y: 8.0,
theta: 0.0}"
ts ros2 service call /turtle1/set_pen turtlesim/srv/SetPen "{r: 255, g: 0, b: 0, width: 4, 'off':
0}"
ts ros2 service call /clear std_srvs/srv/Empty
ts ros2 service call /kill turtlesim/srv/Kill "{name: turtle2}"
```

6. T2 - action. Turn to 1.57 rad (90 degrees) and wait for the result.

```
function ts { pixi run cmd /c scripts\in_upstream.bat turtlesim @args }
ts ros2 action send_goal /turtle1/rotate_absolute turtlesim/action/RotateAbsolute "{theta: 1.57}"
```

7. T2 - a node of the package: `draw_square`. The turtle drives a square (about 20 s). Stop with Ctrl+C.

```
function ts { pixi run cmd /c scripts\in_upstream.bat turtlesim @args }
ts ros2 run turtlesim draw_square
```

```
# Ctrl+C in T1 and T2
```

8. Two simulators and `mimic`. T1: the normal turtlesim. T2: a second one in namespace `/turtlesim2`. T3: `mimic` copies the pose of `turtle1` of the first to the command input of the second.

```
pixi run demo-turtlesim
```

```
function ts { pixi run cmd /c scripts\in_upstream.bat turtlesim @args }
ts ros2 run turtlesim turtlesim_node --ros-args -r __ns:=/turtlesim2
```

```
function ts { pixi run cmd /c scripts\in_upstream.bat turtlesim @args }
ts ros2 run turtlesim mimic --ros-args -r input/pose:=/turtle1/pose -r
output/cmd_vel:=/turtlesim2/turtle1/cmd_vel
```

Now move the first turtle (T4); the second one follows it exactly:

```
function ts { pixi run cmd /c scripts\in_upstream.bat turtlesim @args }
ts ros2 topic pub --once /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0}, angular: {z:
1.8}}"
Start-Sleep 3
ts ros2 topic echo /turtlesim2/turtle1/pose --no-daemon --spin-time 5 --once
```

```
# Ctrl+C in T1, T2, T3
```

Observations (recorded)

Item	Value
Start pose	(5.544, 5.544), theta 0
After `pub --once` (2.0, 1.8)	(6.60, 6.94), theta 1.81
`spawn`	reply `name='turtle2'` (a second `spawn` with the same name is refused - checked by `test-turtlesim`)
`rotate_absolute 1.57`	about 100 feedback messages (`remaining` from 1.57 to 0), result `delta` about -1.55, status SUCCEEDED
`mimic`	the pose of `/turtlesim2/turtle1` equals that of `/turtle1`

Verification

```
pixi run test-turtlesim
```

15 checks on the source-built package: the overlay shadows the pixi one; `cmd_vel` of 2.0 for one second moves 2.02 units; both teleport services; `spawn` (and refusal of a duplicate) and `kill`; `set_pen`; the `rotate_absolute` action; and `mimic` across two instances.

Questions

1. Why does the turtle move only for one second after a single `topic pub --once`?
2. Which interface type would you use to (a) ask for the position once, (b) rotate to an angle and be told when done, (c) stream velocities? What does turtlesim use?
3. Why must the second simulator run in a namespace? What would `mimic`'s two remaps look like if both simulators used namespaces?
4. How does `ros2 pkg prefix` prove which build you run?
5. *(Extension)* Write a node that draws a circle of radius 2 using only `/turtle1/cmd_vel`.

If it does not work

Symptom	Fix
`upstream_ws\turtlesim is not built`	`pixi run build-turtlesim`
Window is black / not drawn	wait for start-up; close other turtlesim windows from earlier labs
`Failed to populate field` in a service call	put quotes around the whole `{...}` and quote the key `'off'`

Lab U3 - The `urdf_tutorial` robots (R2-D2 style, eight steps)

Aim. Build the official `urdf_tutorial` package from its repository and look at its eight example robots, from one cylinder to a macro-generated robot, to see how each URDF/xacro feature changes what RViz2 draws.

Time. 45 min. **Needs.** Only this folder. (The package is pure Python/XML: no compiler.) The repository is downloaded by `pixi run build-urdf`. Source: `upstream\urdf_tutorial-ros2` (branch `ros2`), built into `upstream_ws\urdf`.

This folder is a complete project

Everything this lab needs is in this folder. Nothing from the other labs is required.

Requirements: Windows 10/11, PowerShell, pixi (no administrator rights needed; install steps are in README-SETUP.md), and internet access once, to download `ros/urdf_tutorial` from GitHub.

In this folder	
`upstream\`	the original repository (downloaded by the first build task, pinned commit); `upstream_ws\` its build
`scripts\`	helper scripts and the automated tests
`pixi.toml`	the environment and every `pixi run ...` command; `README.md` this manual; `tools\replay.ps1` re-runs the manual's commands as a self-test

Set it up once. `pixi install` puts ROS 2 Jazzy into `C:\rlabs` (a few GB; pixi's package cache makes the second lab on the same machine fast). Open PowerShell in this folder and run:

```
pixi install
pixi run verify
```

Run one lab at a time, and everything from this folder.

Theory

File	New feature
`01-myfirst.urdf`	one link with a cylinder
`02-multipleshapes.urdf`	two links, one **fixed joint**
`03-origins.urdf`	joint **origin** (`xyz` , `rpy`): where the child sits
`04-materials.urdf`	**materials** (colours)
`05-visual.urdf`	different shapes, **meshes** for the head
`06-flexible.urdf`	**movable joints**: continuous, revolute, prismatic - sliders appear
`07-physics.urdf`	`collision` and `inertial` tags (needed by a physics simulator)
`08-macroed.urdf.xacro`	**xacro**: constants, macros, math - same robot, much shorter

`display.launch.py` starts `robot_state_publisher` (URDF -> TF), `joint_state_publisher_gui` (sliders for the movable joints) and RViz2. The robot is chosen with the launch argument `model:=urdf/<file>`.

Procedure

Shortcut for commands that need this package (re-type in each new terminal):

```
function urdf { pixi run cmd /c scripts\in_upstream.bat urdf @args }
```

1. Build.

```
pixi run build-urdf
```

2. T1 - the first robot (a single blue cylinder; fixed frame `base_link`).

```
pixi run demo-urdf
```

3. T2 - what was published.

```
pixi run ros node list --no-daemon --spin-time 5
pixi run ros topic list --no-daemon --spin-time 5
```

4. T1 - the movable robot. Stop T1 (Ctrl+C), then start the sixth robot. Move the sliders: the head turns, the gripper opens, the arm slides.

```
# Ctrl+C in T1
```

```
pixi run demo-urdf model:=urdf/06-flexible.urdf
```

```
pixi run ros topic echo /joint_states --no-daemon --once --spin-time 5
```

(Only the movable joints are in `/joint_states`; the fixed ones never change.)

5. T1 - the macro version. Compare 07 and 08: same robot, one file written with xacro.

```
# Ctrl+C in T1
```

```
pixi run demo-urdf model:=urdf/08-macroed.urdf.xacro
```

```
pixi run ros topic echo /robot_description --no-daemon --once --spin-time 5 --field data
```

Count the lines of the two source files, and expand the xacro to plain URDF to see what the macros produce:

```
(Get-Content upstream\urdf_tutorial-ros2\urdf\07-physics.urdf).Count    # 417
(Get-Content upstream\urdf_tutorial-ros2\urdf\08-macroed.urdf.xacro).Count    # 237
pixi run xacro upstream\urdf_tutorial-ros2\urdf\08-macroed.urdf.xacro -o r2d2.urdf
pixi run check_urdf r2d2.urdf
Remove-Item r2d2.urdf
```

```
# Ctrl+C in T1
```

6. Try the others. Run `pixi run demo-urdf model:=urdf/0N-....urdf` for 01-05 and 07. Before each, predict what RViz2 will draw.

Observations (recorded)

Item	Value
Nodes with `01-myfirst`	`/robot_state_publisher`, `/rviz`; topics `/robot_description`, `/tf_static` (no `/joint_states`: the robot has no movable joint). With `06-flexible` the slider node `/joint_state_publisher` also appears and `/joint_states` is published
`06-flexible` joints in `/joint_states`	`head_swivel`, `gripper_extension`, `left_gripper_joint`, `right_gripper_joint` and the four wheel joints
Source size	`07-physics.urdf` 417 lines, `08-macroed.urdf.xacro` 237 lines (the robot is named `macroed`); the xacro expands to a URDF with root link `base_link`

Verification

```
pixi run test-urdf
```

17 checks: for each of the eight robots the package's own `display.launch.py` is started; the published `/robot_description` equals the source file (after xacro: same link and joint counts) and every link resolves in TF.

Questions

1. Which joint types appear in `06-flexible.urdf` and which of them gets a slider?
2. In `03-origins.urdf`, what is the difference between the joint `origin` and the visual `origin`?
3. How many lines does the macro version save (417 vs 237), and which repeated block does the macro replace?
4. Why does `07-physics.urdf` need `inertial` and `collision` tags when RViz2 does not?
5. *(Extension)* Add a camera link to `08-macroed.urdf.xacro` using the same macro style (edit a *copy*, not the clone).

If it does not work

Symptom	Fix
`malformed launch argument 'model:'`	you are using an older `scripts\in_upstream.bat`; update it (it must read `%*`)
`package 'urdf_tutorial' not found`	`pixi run build-urdf`
RViz2 grey	set *Fixed Frame* to `base_link`

Lab U4 - Plug your own planner into Nav2 (Robotics & ROS 2 Essentials, exercise 5)

Aim. The *Robotics & ROS 2 Essentials* repository contains a C++ **Nav2 planner plugin** that does no planning itself: whenever Nav2 needs a path it asks the ROS **service** `/create_plan`. You write that service in Python (here: a straight line), load the plugin into real Nav2 and watch DiffBot follow *your* path.

Time. 60 min. **Needs.** Only this folder, plus a C++ compiler (Visual Studio Build Tools, C++ workload) for the plugin. The Essentials repository is downloaded by `pixi run prepare-essentials`. Source: `upstream\robotics_essentials_ros2` (never modified); a patched copy is built into `upstream_ws\essentials`.

This folder is a complete project

Everything this lab needs is in this folder. Nothing from the other labs is required.

Requirements: Windows 10/11, PowerShell, pixi (no administrator rights needed; install steps are in README-SETUP.md), and Visual Studio Build Tools (C++ workload); internet access once, to download the Essentials repository from GitHub.

In this folder	
`ws\src\`	the ROS 2 packages; `ws\install\` appears after `pixi run build`
`upstream\`	the original repository (downloaded by the first build task, pinned commit); `upstream_ws\` its build
`scripts\`	helper scripts and the automated tests
`code\`	reference solutions / helper programs for this lab
`pixi.toml`	the environment and every `pixi run ...` command; `README.md` this manual; `tools\replay.ps1` re-runs the manual's commands as a self-test

Set it up once. `pixi install` puts ROS 2 Jazzy into `C:\rlabs` (a few GB; pixi's package cache makes the second lab on the same machine fast). Open PowerShell in this folder and run:

```
pixi install
pixi run verify
pixi run build
```

(`pixi run build` compiles this lab's packages into `ws\install`; run it again after every edit in `ws\src`.)

Run one lab at a time, and everything from this folder.

Theory

Nav2 loads its global planner as a **plugin** (a C++ class loaded by name at start-up). `custom_nav2_planner/CustomPlanner` implements Nav2's planner interface by forwarding the request:

```
Nav2 planner_server --(plugin CustomPlanner)--> service /create_plan
(create_plan_msgs/srv/CreatePlan) --> your node
        start, goal (PoseStamped)  ------------------------------------------------>  returns
nav_msgs/Path
```

`CreatePlan.srv`: request `geometry_msgs/PoseStamped start, goal`; response `nav_msgs/Path path`. Nav2's controller then follows whatever path you return. Nav2 re-plans about once per second, so your service is called repeatedly while the robot drives.

The repository targets ROS 2 Humble. In Jazzy the plugin interface gained a third argument (`cancel_checker`); `scripts\prepare_essentials.ps1` copies the repository's two packages and adds exactly that argument (two lines, see `patches\essentials-jazzy.patch`).

Procedure

1. Prepare and build the repository's two packages (`create_plan_msgs`, `custom_nav2_planner`).

```
pixi run prepare-essentials
pixi run build-essentials
```

2. Read the service you will implement.

```
pixi run cmd /c scripts\in_upstream.bat essentials ros2 interface show
create_plan_msgs/srv/CreatePlan
```

3. T1 - start real Nav2 with the plugin instead of NavFn. `COURSE=1` makes the helper load the course workspace in addition to the repository's overlay. Wait for `Managed nodes are active`; the log must say that the planner is `custom_nav2_planner/CustomPlanner`.

```
$env:COURSE = "1"
pixi run cmd /c scripts\in_upstream.bat essentials ros2 launch diffbot_bringup nav2.launch.py
planner:=custom rviz:=true
```

4. T2 - start your service. Read `code\create_plan_server.py` first: `on_create_plan()` returns a point every 0.1 m on the line start -> goal. It prints one line for every request it gets.

```
$env:COURSE = "1"
pixi run cmd /c scripts\in_upstream.bat essentials python code\create_plan_server.py
```

5. T3 - send a goal. (2.2, 3.0) is free space, 2.3 m from the start, so a straight line is safe. In T2 you see `plan (...) -> (2.20, 3.00)` about once per second, each with fewer points as the robot gets closer.

```
pixi run goto 2.2 3.0
```

In RViz2 the path (`/plan`) is a perfectly straight line, unlike Nav2's own NavFn planner (`planner:=navfn`), which bends the path around the walls.

```
# Ctrl+C in T1 and T2
```

6. The exercise. Replace the body of `on_create_plan()` with a planner that avoids the walls (for example run A* on the map from the `/map` topic) and send `pixi run goto 8.5 6.5`. Nav2 needs no change.

Observations (recorded)

Item	Value
Planner plugin in the Nav2 log	`Created global planner plugin GridBased of type custom_nav2_planner/CustomPlanner`
Service calls during the drive	11 (`Got path as a service result` in the Nav2 log)
Goal (2.2, 3.0)	reached; last feedback 0.35 m, then `REACHED the goal`; about 20 s
Path published on `/plan`	exact straight line from start to goal (deviation under 3 cm)

Verification

```
pixi run test-essentials
```

10 checks: the plugin is loaded and NavFn is not; our service was called at least once; the published plan is the straight line (max deviation < 3 cm); the robot arrives within 0.15 m of (2.2, 3.0), driving about 2.2 m for 2.3 m of straight line, with no collision. Recorded: 10/10.

Questions

1. Why does the plugin return our path unchanged - which Nav2 component then makes sure the robot does not hit a wall?
2. Why is a service (not a topic) the right interface between the plugin and your node?
3. What happens at `goto 8.5 6.5` with the straight-line planner? Try it, and explain what the local costmap and controller do.
4. The repository's plugin source had to be changed in two lines for Jazzy. What does `cancel_checker` allow a planner to do?
5. *(Extension)* Return a path that first goes 1 m sideways (a dog-leg) and verify in RViz2 that Nav2 follows it.

If it does not work

Symptom	Fix
`goto` says "goal rejected"	Nav2 not active yet; wait for `Managed nodes are active`
Nav2 log says `GridBased ... NavfnPlanner`	you started without `planner:=custom`
`No module named create_plan_msgs`	T2 was started without the helper (`scripts\in_upstream.bat essentials`) or without `COURSE=1`
The robot does not move	T2 is not running: the plugin cannot get a path without your service

Lab U5 - Gazebo, odometry, SLAM and Nav2 in the Essentials Docker image

Aim. Run the exercises of *Robotics & ROS 2 Essentials* in **its own Docker image** (Ubuntu 22.04, ROS 2 Humble, Gazebo Fortress, the **Andino** robot, Nav2, slam_toolbox) - something Windows cannot do natively - and check four things stage by stage: the simulated robot obeys `/cmd_vel`, your wheel odometry is right, `slam_toolbox` builds a map, and a planner plugin works inside Nav2.

Time. 30 min (+ tens of minutes the first time, for the 6.5 GB image). **Needs.** Only this folder, plus **Docker Desktop** (WSL 2 backend). The repository is downloaded by `pixi run build-essentials-docker`. Docker Desktop must be running (about 12 GB RAM for its VM, 10 GB free disk).

This folder is a complete project

Everything this lab needs is in this folder. Nothing from the other labs is required.

Requirements: Windows 10/11, PowerShell, pixi (no administrator rights needed; install steps are in README-SETUP.md), and Docker Desktop with the WSL 2 backend (about 12 GB RAM for its VM, 10 GB disk); internet access for the first build.

In this folder	
`scripts\docker\`	the image build and the four test stages; `upstream\` the repository (downloaded by the build task)
`scripts\`	helper scripts and the automated test
`pixi.toml`	the environment and every `pixi run ...` command; `README.md` this manual; `tools\replay.ps1` re-runs the manual's commands as a self-test

Set it up once. `pixi install` provides `git` (the only tool the scripts need). Open PowerShell in this folder and run:

```
pixi install
```

Run one lab at a time, and everything from this folder.

Theory

- A Docker **image** is a frozen Linux installation; a **container** is one running copy. Everything the exercises need is inside the image, so nothing is installed on Windows.
- Gazebo normally opens a window. Here it runs **headless** on a virtual screen (Xvfb, added by a 20 MB layer on top of the repository's image). Nothing is drawn on your desktop.
- Software rendering makes Gazebo run at about **0.5 x real time**. Every test therefore measures time with the simulated clock `/clock`, never with a wall clock.
- Andino: wheel radius 0.033 m, separation 0.137 m, a 720-ray lidar. Wheel odometry is the usual differential-drive maths (`ds = (dL + dR)/2`, `dtheta = (dR - dL)/L`) with these two numbers.

Stage	Exercise	What it proves
`sim`	0-1	Gazebo loads, `/scan` `/odom` `/joint_states` `/clock` publish, `/cmd_vel 0.2` for 4 simulated s moves the robot 0.80 m

Stage	Exercise	What it proves
`odometry`	4	odometry from the wheels matches the commanded motion; Gazebo's *physical* pose differs, because simulated wheels slip
`slam`	2	`slam_toolbox` builds a map while the robot turns and drives; `map_saver_cli` writes a PNG + YAML
`nav`	5	the repository's C++ plugin runs inside Andino's Nav2 and gets its path from a Python `create_plan` service

Procedure

1. Check Docker.

```
docker --version
docker info --format "Server: {{.ServerVersion}}"
```

2. Build the images (the repository's own `docker/Dockerfile` plus the thin headless layer; seconds when cached, tens of minutes on the first run).

```
pixi run build-essentials-docker
```

3. Look inside the image. A throw-away container lists the ROS packages that belong to the robot.

```
docker run --rm robotics_essentials_ros2:headless ros2 pkg list | Select-String
"andino|slam_toolbox"
```

4. Run the stages one at a time (each starts a container, runs its checks, prints `PASS` / `FAIL` lines and removes the container).

```
powershell -NoProfile -ExecutionPolicy Bypass -File scripts\docker\run_essentials_docker.ps1 -Stage
sim
```

```
powershell -NoProfile -ExecutionPolicy Bypass -File scripts\docker\run_essentials_docker.ps1 -Stage
odometry
```

```
powershell -NoProfile -ExecutionPolicy Bypass -File scripts\docker\run_essentials_docker.ps1 -Stage
slam
```

```
powershell -NoProfile -ExecutionPolicy Bypass -File scripts\docker\run_essentials_docker.ps1 -Stage
nav
```

5. Look at the map that SLAM built (opens in your picture viewer; white = free, black = walls, grey = unknown).

```
Invoke-Item build\docker_out\andino_map.png
```

Observations (recorded; the real-time factor depends on your CPU)

Stage	Recorded result
`sim`	6 checks; real-time factor 0.65; `/scan` 720 rays; **0.80 m** in 4 simulated seconds
`odometry`	6 checks; our odometry off by 0.003 m / 0.000 rad from the commanded motion; Gazebo's physical pose moved 0.87 m and differs from the ideal (slip)
`slam`	7 checks; map about 395 x 301 cells at 0.05 m (44 000 free, 560-600 occupied); `andino_map.png` (13.7 kB) + `.yaml` written to `build\docker_out`
`nav`	13 checks; plugin `custom_nav2_planner/CustomPlanner` loaded, goal 1.6 m ahead reached (0.23 m off), `create_plan` called 14 times, plan = straight line (0.000 m deviation)

Verification (all four stages in one command, 7-8 minutes)

```
pixi run test-essentials-docker
```

Recorded: 32 of 32 checks.

Questions

1. Why is Gazebo's own `/odom` farther from the commanded motion than your wheel odometry? Which of the two says where the robot *really* is?
2. Why do the tests use `/clock` instead of `Start-Sleep`?
3. What would be different if the container had no virtual display? (Gazebo's GUI could not start.)
4. Which Docker option would you add to see Gazebo's window on Linux (hint: `DISPLAY`, X11 socket)? Why is that not possible directly on Windows?
5. *(Extension)* Open `scripts\docker\run_stage.sh`: which command starts Gazebo, and which lines wait for the robot to be ready?

If it does not work

Symptom	Fix
`docker : The term 'docker' is not recognized`	install / start Docker Desktop; `docker desktop start`
`Unable to locate package` while building	the repository was cloned with Windows line endings; use the LF clone made by `build-essentials-docker`
A stage hangs	Docker Desktop has too little memory (Settings -> Resources); the stage is stopped after 20 min